

CONTROLS — SYSTEMS —

— Project 13 — UAV Attitude Control System



ROLL	0.2°
PITCH	-0.3°
YAW	1.1°



— Designed, Developed and Documented by —

Zohaib Imtiaz

MATLAB R2024b • Control System Toolbox • Aerospace Engineering

Copyright © 2026 Zohaib Imtiaz. All Rights Reserved.

This report is the original work of the author and has been prepared for academic purposes. No part of this document may be reproduced, distributed, or transmitted in any form without proper acknowledgment of the author, except where permitted for educational or research purposes.

Abstract

This project develops and stress-tests a complete UAV attitude-control architecture in Simulink, using a physically grounded example vehicle (Black Hornet Nano class: 33 g mass, 12 cm rotor diameter, 10 cm fuselage length). Building on the classical and modern control techniques established in Projects 01–12, this project integrates them into a single closed-loop system: single-axis rotational plants, Euler cross-axis coupling, a realistic noisy IMU sensor model, three independently-tuned steady-state Kalman filters, closed-loop estimated-state feedback, external disturbance rejection, and a formal operating-envelope and stress-testing analysis.

Across eight phases, the investigation follows a consistent predict-then-verify methodology: every major analytical prediction — critical damping values, coupling-torque ratios, sensor noise levels — was independently confirmed by simulation before being accepted. The central finding is that roll and yaw remain stable and within acceptance criteria across the entire tested envelope (5° – 180° commanded attitude, $1\times$ – $64\times$ disturbance severity), while pitch exhibits a precise, traceable performance limitation — caused by an under-tuned rate estimator combined with pitch's structural sensitivity to coupling — that degrades performance from the smallest tested angle onward and transitions to genuine instability only at attitude magnitudes that also exceed the underlying model's small-angle validity. This is characterized as a targeted, well-understood limitation in a single estimator, not a failure of the control architecture as a whole.

Table of Contents

Abstract	iii
List of Tables	vi
Table of Figures	vii
1 Objectives	1
2 Introduction.....	2
3 System Description	3
4 Assumptions.....	4
5 Phase 3 — Single-Axis Foundations	5
6 Phase 4 — Axis Coupling.....	7
7 Phase 5 — Sensor Modeling and Kalman Estimation	10
7.1 Sensor Model	10
7.2 Kalman Filter	12
8 Phase 6 — Closed-Loop Estimate Feedback and Disturbance Rejection.....	15
8.1 Closing the Loop on Estimated State.....	15
8.2 Disturbance Rejection.....	16
8.3 Severity Sweep ($1\times$ – $8\times$ Baseline Disturbance)	18
8.4 8.4 Simultaneous Multi-Axis Disturbance on the Coupled Model.....	18
9 Phase 7 — Performance Evaluation and Operating Envelope.....	19
10 Phase 8 — Stress Testing to the Failure Boundary.....	20
10.1 Pitch's Divergence Boundary Is Angle-Driven.....	21
10.2 Severity Matters More at Small Angles.....	21
10.3 Roll's Single Failure Point	21
10.4 Yaw's Complete Robustness	22
11 Aerospace Relation	23
12 Key Engineering Conclusions.....	24
13 Model Limitations and Assumptions Summary	25

14	Future Work	25
15	Appendix.....	26
15.1	MATLAB Code	26

List of Tables

<i>Table 1 UAV Parameters</i>	<i>3</i>
<i>Table 2 Per-axis inertia, theoretical and simulated critical damping, and resulting rise time.....</i>	<i>5</i>
<i>Table 3 Euler cross-coupling torque as a percentage of control torque at 5°, 10°, and 57.3°.....</i>	<i>7</i>
<i>Table 4 Coupling coefficient sign per axis and its physical effect on that axis's response.</i>	<i>8</i>
<i>Table 5 Per-axis raw rate signal-to-noise ratio, optimal process-noise tuning, and resulting rate-filter improvement.</i>	<i>12</i>
<i>Table 6 Peak deviation and recovery behaviour per axis under true-state versus estimate feedback.</i>	<i>16</i>
<i>Table 7 Severity sweep results: steady-state RMS chatter at 1× and 8× baseline disturbance, and the first crossing into unstable chatter.....</i>	<i>18</i>
<i>Table 8 Simultaneous multi-axis disturbance results on the coupled model at 10° and 57.3°.</i>	<i>18</i>
<i>Table 9 Acceptance criteria thresholds applied throughout Phases 7 and 8.</i>	<i>19</i>
<i>Table 10 Pitch performance, coupled-only versus coupled-plus-disturbance, at each tested commanded angle.</i>	<i>20</i>
<i>Table 11 Summary of divergence conditions identified across all 56 tested angle-severity combinations. Yaw does not appear — it did not diverge under any tested condition.....</i>	<i>22</i>
<i>Table 12 Combined verdict per axis across nominal, aggressive, and extreme operating regimes.....</i>	<i>22</i>

Table of Figures

Figure 1 Black Hornet Nano	2
Figure 2 Body-axis convention for the Black Hornet Nano	3
Figure 3 Single-axis Simulink architecture (roll)	5
Figure 4 Overshoot and settling time versus b.....	6
Figure 5 Coupled Simulink model wiring	7
Figure 6 Single-axis sanity check: roll-only command showing zero response on the uncommanded pitch and yaw axes.	8
Figure 7 Simultaneous three-axis step response at 57.3°, showing pitch's larger overshoot relative to roll and yaw.	9
Figure 8 IMU sensor model subsystem: white-noise injection on all six true states	10
Figure 9 True versus measured roll angle (ϕ) and roll rate (Kalman Filter Design and Tuning)	11
Figure 10 Kalman Filter Block	12
Figure 11 Kalman filter Simulink implementation	13
Figure 12 True, measured, and Kalman-estimated angle	13
Figure 13 Full closed-loop architecture.	15
Figure 14 Disturbance pulse construction.....	16
Figure 15 Roll disturbance rejection.....	17
Figure 16 Pitch disturbance response	17

1 Objectives

The primary objective of this project was to move beyond the isolated, single-technique studies of Projects 01–12 and construct a complete, integrated UAV attitude-control system — one that combines plant modeling, rigid-body axis coupling, realistic sensor noise, Kalman state estimation, closed-loop estimate feedback, and external disturbance rejection into a single Simulink architecture. A further objective was to formally characterize that architecture's operating envelope: not simply whether it works, but precisely under what conditions it works, where it degrades, and where — if anywhere — it becomes genuinely unstable.

Eight phases were pursued in sequence, each phase's design and test conditions building directly on the previous phase's confirmed results, following the same predict-simulate-explain methodology used throughout the broader portfolio.

2 Introduction

Projects 01 through 12 each investigated a single control-engineering tool in isolation — PID tuning, root locus, frequency response, state-space methods, pole placement, LQR, and Kalman filtering — typically applied to a simplified single-axis plant. Real flight-control systems, however, must combine all of these simultaneously: a physically coupled, multi-axis rigid body; imperfect sensors; a state estimator standing between the sensors and the controller; and disturbances the vehicle did not command.

Project 13 is the first project in the series to build that complete system. The vehicle chosen — a Black Hornet Nano-class nano-helicopter — was selected specifically because its very small inertia produces unusually large angular rates for modest control torques, which surfaces interactions between coupling, sensor noise, and estimation that would be far less visible on a larger, slower vehicle.



Figure 1 Black Hornet Nano

3 System Description

The vehicle is modeled as a rigid body with three independent rotational degrees of freedom — roll (ϕ), pitch (θ), and yaw (ψ) — each driven by its own commanded torque and each subject to Euler cross-axis coupling from the other two. Attitude and rate measurements are corrupted by sensor noise representative of a small MEMS IMU, and true state is never directly available to the controller from Phase 5 onward — only a Kalman-filtered estimate is.

Table 1 UAV Parameters

Parameter	Value
Vehicle class	Black Hornet Nano (coaxial nano-helicopter)
Mass	33 g (0.033 kg)
Rotor diameter	12 cm (0.12 m)
Fuselage length	10 cm (0.10 m)
Roll inertia (I_{xx})	$5.94 \times 10^{-5} \text{ kg} \cdot \text{m}^2$ (solid-cylinder approximation)
Pitch inertia (I_{yy})	$2.76 \times 10^{-5} \text{ kg} \cdot \text{m}^2$ (thin-rod approximation)
Yaw inertia (I_{zz})	$8.70 \times 10^{-5} \text{ kg} \cdot \text{m}^2$ (perpendicular axis theorem)

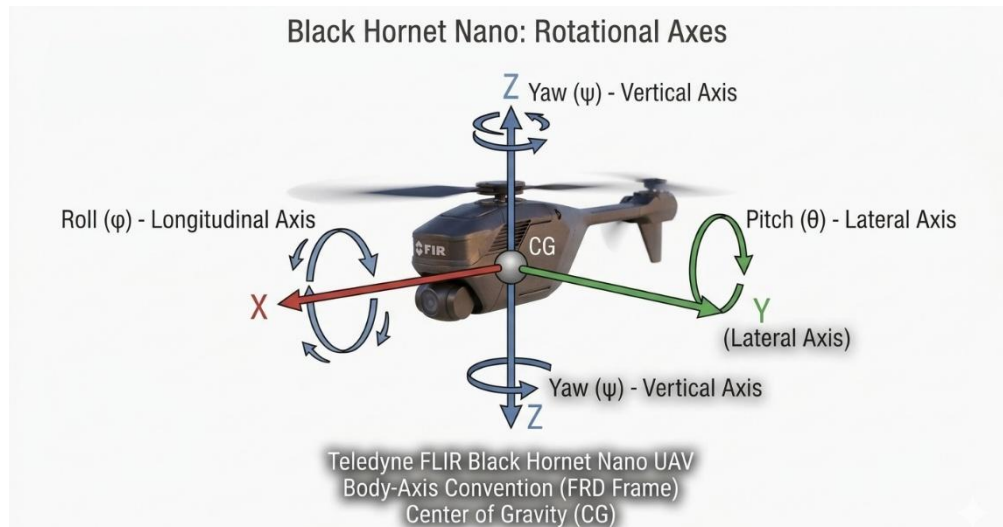


Figure 2 Body-axis convention for the Black Hornet Nano

4 Assumptions

1. Each axis is modeled as a single rotational mass-damper ($I \cdot \ddot{\theta} + b \cdot \dot{\theta} = \tau$); no aerodynamic damping data was available for the real vehicle, so damping was determined by theoretical critical-damping design, not measurement.
2. Inertia values are geometric approximations (cylinder, rod, perpendicular-axis theorem), not measured properties of an actual Black Hornet Nano airframe.
3. Rotor gyroscopic coupling (a separate mechanism from Euler rigid-body coupling) was identified as conceptually relevant to this vehicle's coaxial contra-rotating rotors, but was not modeled or quantified — only Euler cross-coupling terms are represented.
4. Sensor noise levels are drawn from MPU-9250 datasheet specifications, a representative small consumer MEMS IMU, not a measured characteristic of any specific hardware used on this vehicle. Sensor bias/drift was excluded as a first-pass simplification.
5. Kalman filters are linear, steady-state, single-axis designs; none incorporate the Euler coupling terms in their internal model — this omission is the direct subject of the project's central finding, not an oversight.
6. Attitude commands and coupling analysis beyond roughly 90–120° exceed the small-angle assumptions ($\sin \theta \approx \theta$) underlying the linearized coupling model; results at 120–180° are reported as simulation-domain findings, not physical predictions for the real vehicle.

5 Phase 3 — Single-Axis Foundations

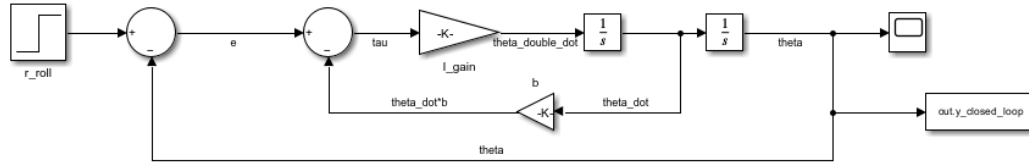


Figure 3 Single-axis Simulink architecture (roll)

Pitch and yaw use an identical structure with their own inertia and damping values.

Each axis's inertia was derived from a simplifying geometric approximation and cross-checked where possible (e.g., roll's cylinder estimate against a thin-rod estimate, both landing in the same order of magnitude). Each axis was then modeled as a rotational mass-damper under unity-feedback P-control, and a systematic damping (b) sweep was run per axis, comparing simulated step-response behavior against the theoretical critical-damping prediction $b_{\text{critical}} = 2\sqrt{I \cdot k}$.

Table 2 Per-axis inertia, theoretical and simulated critical damping, and resulting rise time.

Axis	Inertia (kg·m ²)	b_critical (theory)	b_critical (simulated)	Rise Time
Roll	5.94×10^{-5}	0.0154	0.0154	0.026 s
Pitch	2.76×10^{-5}	0.0105	0.0105	0.018 s (fastest)
Yaw	8.70×10^{-5}	0.0187	0.01865	0.038 s (slowest)

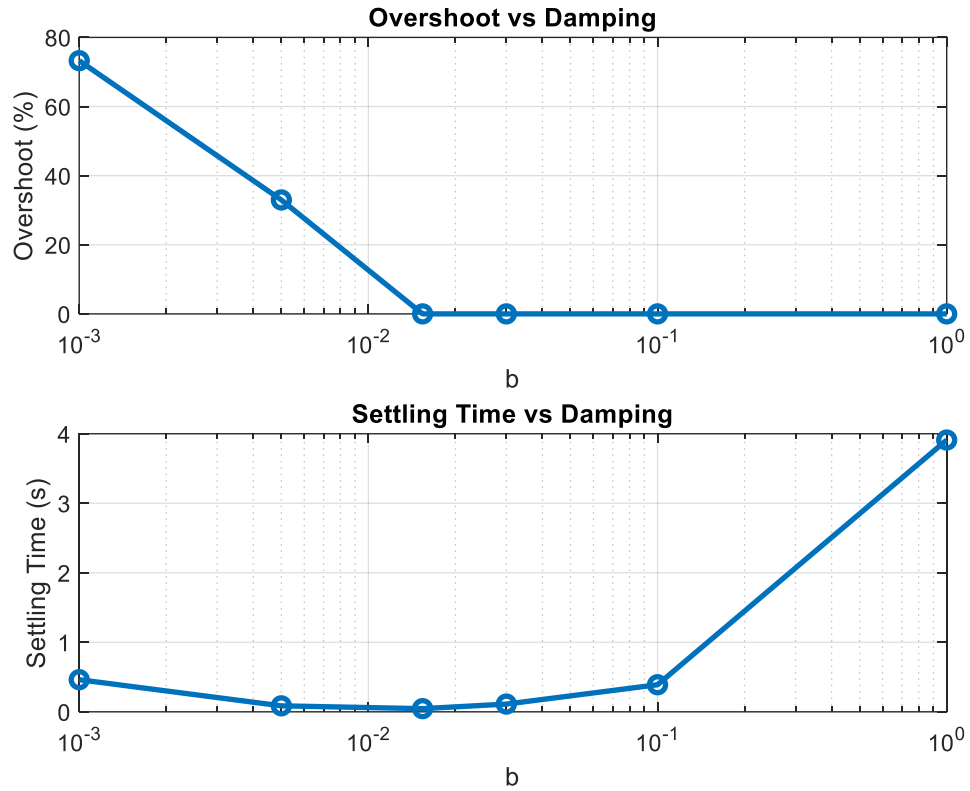


Figure 4 Overshoot and settling time versus b

Every axis's theoretically predicted critical damping matched its simulated result to within roughly 1% — a pattern of prediction-then-verification that recurs throughout the project. Because the rotational plant already contains a free integrator, unity-feedback P-control alone produces zero steady-state error, and critical damping eliminates overshoot; no PID compensation was required on any axis.

6 Phase 4 — Axis Coupling

Rigid-body rotational dynamics are inherently coupled through Euler's equations of motion. For each axis, the coupling term takes the form $(I_j - I_k) \cdot (\text{rate}_j) \cdot (\text{rate}_k)$ — a product of the other two axes' angular rates, scaled by the difference of their inertias. This coupling torque was estimated analytically at three commanded angles and compared against each axis's own control torque.

Table 3 Euler cross-coupling torque as a percentage of control torque at 5°, 10°, and 57.3°.

Step size	Roll coupling	Pitch coupling	Yaw coupling
5°	0.83%	0.26%	0.60%
10°	1.65%	0.53%	1.19%
57.3° (1 rad)	9.48%	3.03%	6.83%

A coupled Simulink model was then built, cross-wiring each axis's angular rate into the other two axes' dynamics. Single-axis sanity checks confirmed a mathematically necessary property: commanding one axis alone, with the other two starting at rest, produces exactly zero coupling effect on the other axes, since the coupling term is a product of two rates and one of them is identically zero throughout.

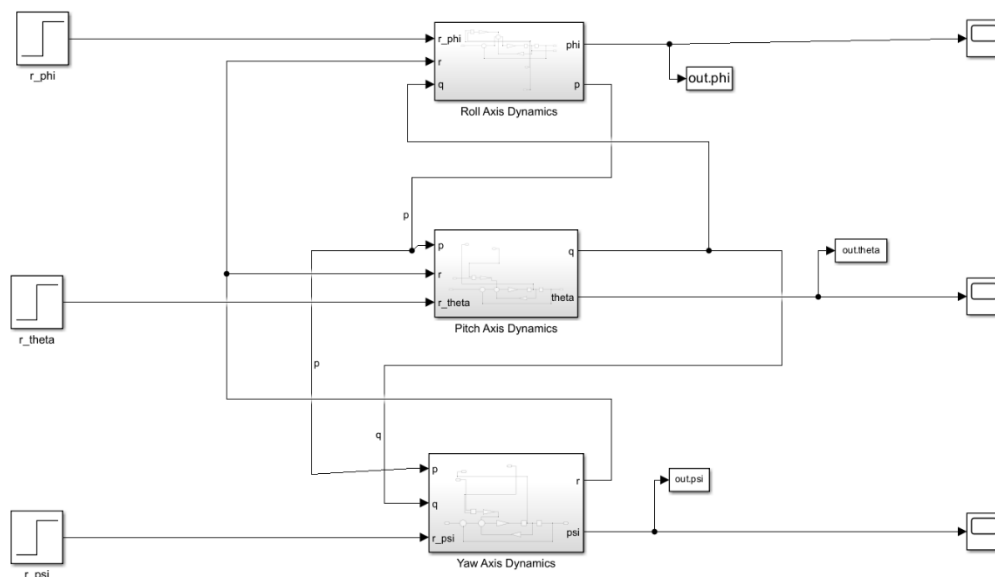


Figure 5 Coupled Simulink model wiring

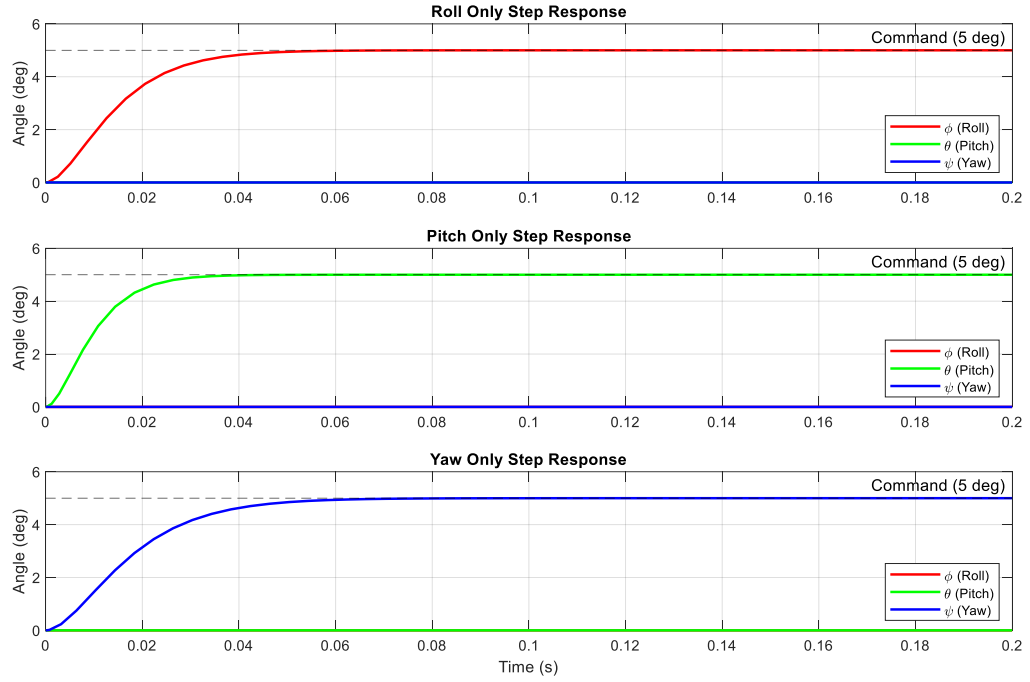


Figure 6 Single-axis sanity check: roll-only command showing zero response on the uncommanded pitch and yaw axes.

Commanding all three axes simultaneously at 57.3° revealed an unexpected result: pitch's overshoot (0.338%) far exceeded roll's and yaw's ($\sim 0.003\%$ each), despite pitch's estimated coupling fraction being the smallest of the three at that angle. This was resolved by examining the sign of each axis's coupling coefficient rather than only its magnitude:

Table 4 Coupling coefficient sign per axis and its physical effect on that axis's response.

Axis	Coefficient	Value	Sign	Effect
Roll	$(I_{\text{pitch}} - I_{\text{yaw}})$	-5.94×10^{-5}	Negative	Opposes motion (damping-like)
Pitch	$(I_{\text{yaw}} - I_{\text{roll}})$	$+2.76 \times 10^{-5}$	Positive	Adds to motion (accelerating)
Yaw	$(I_{\text{roll}} - I_{\text{pitch}})$	$+3.18 \times 10^{-5}$	Positive	Adds to motion (accelerating)

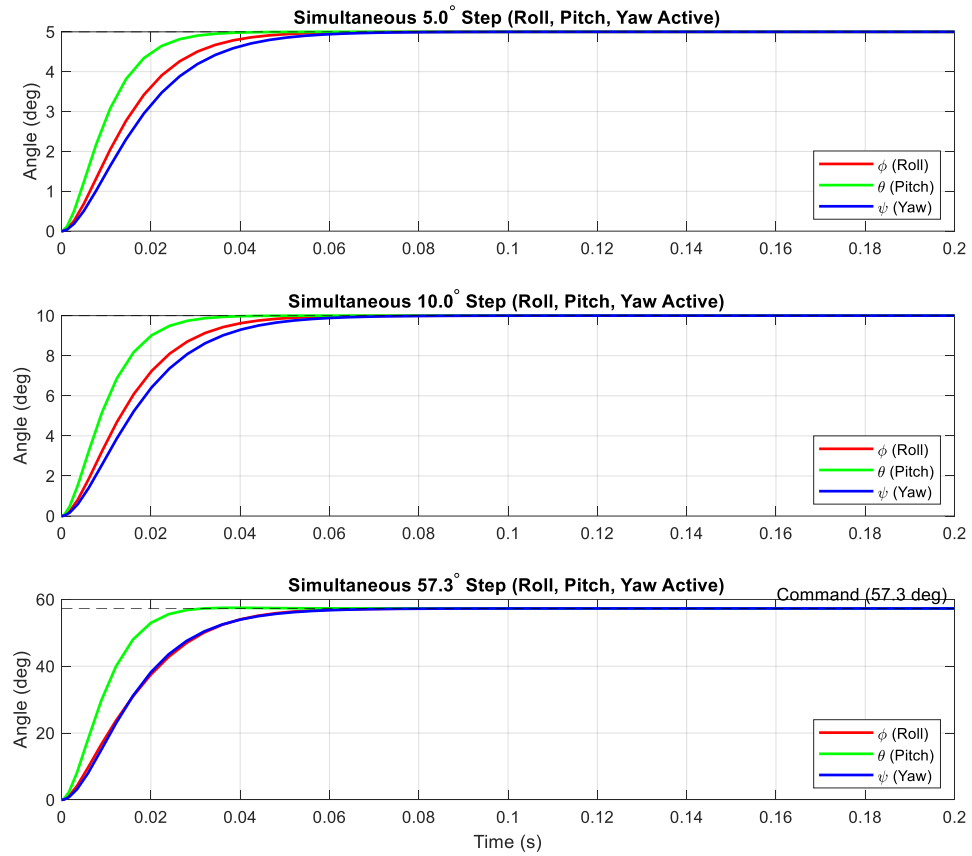


Figure 7 Simultaneous three-axis step response at 57.3°, showing pitch's larger overshoot relative to roll and yaw.

Pitch and yaw share the same coupling sign, but pitch — having the smallest inertia and thus the highest control bandwidth — has the least time margin to absorb an accelerating disturbance before it manifests as overshoot. This is the first appearance of a pattern that recurs through every later phase: pitch's structural properties leave it the most exposed axis to any additional stress.

7 Phase 5 — Sensor Modeling and Kalman Estimation

7.1 Sensor Model

A six-channel IMU sensor model was built, adding independent Band-Limited White Noise to all six true states (ϕ , θ , ψ , p , q , r), with noise levels drawn from MPU-9250 datasheet specifications: gyroscope noise density $0.0013 \text{ rad/s}/\sqrt{\text{Hz}}$, and an accelerometer-derived equivalent angle-noise density of approximately $8.46 \times 10^{-4} \text{ rad}/\sqrt{\text{Hz}}$. At a 1 kHz sample rate, this predicts noise standard deviations of approximately 0.0268 rad (angles) and 0.0411 rad/s (rates) — both confirmed by simulated RMSE to within a few percent.

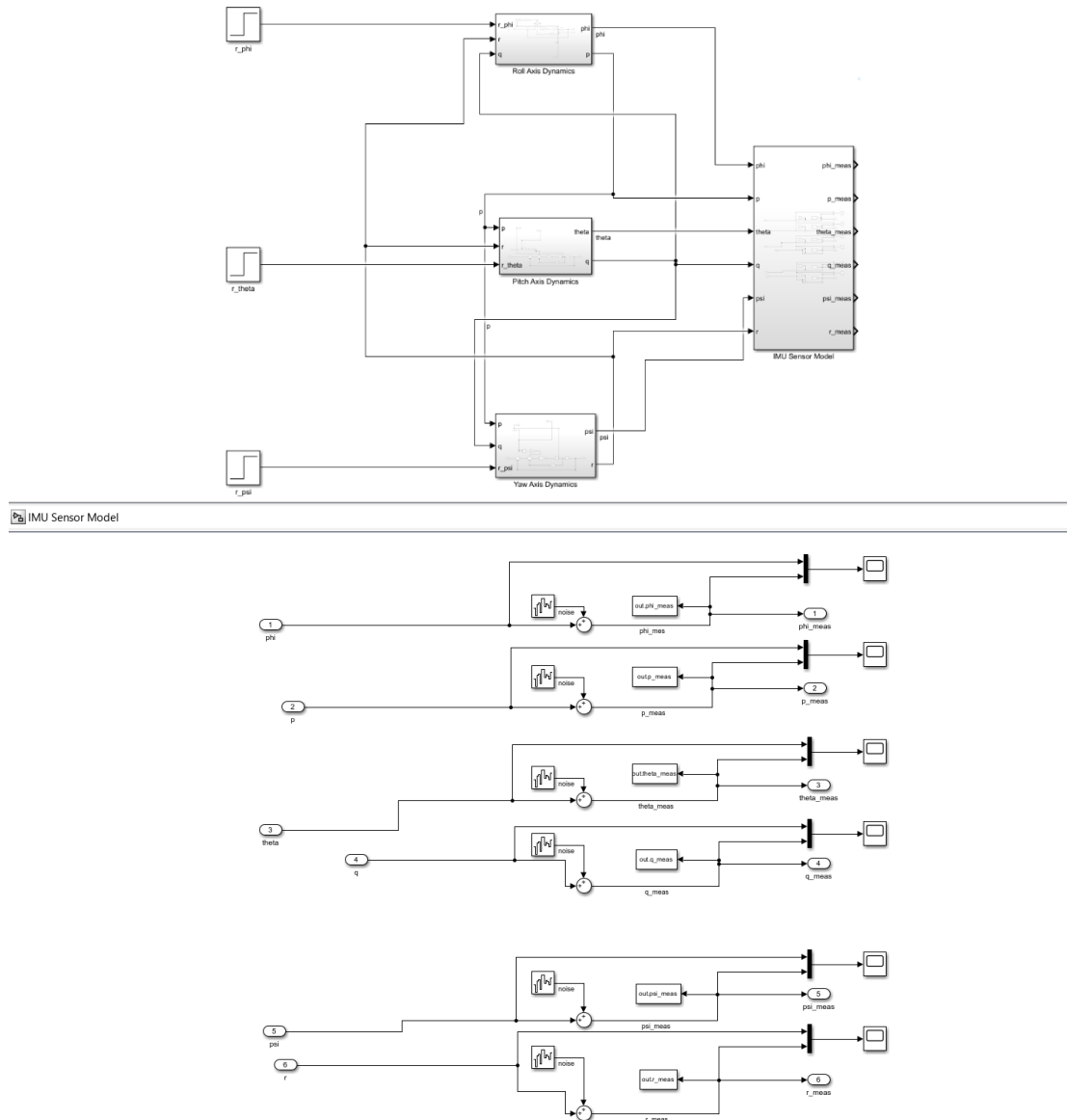


Figure 8 IMU sensor model subsystem: white-noise injection on all six true states

A key finding: angle measurements are 25–50× noisier relative to signal than rate measurements for this vehicle (2.6–2.7% of peak vs. 0.05–0.11% of peak), a direct consequence of the vehicle's unusually large peak angular rates diluting the rate channel's noise floor. This motivated the Kalman filter design: fuse a noisy angle measurement with a comparatively clean rate measurement.

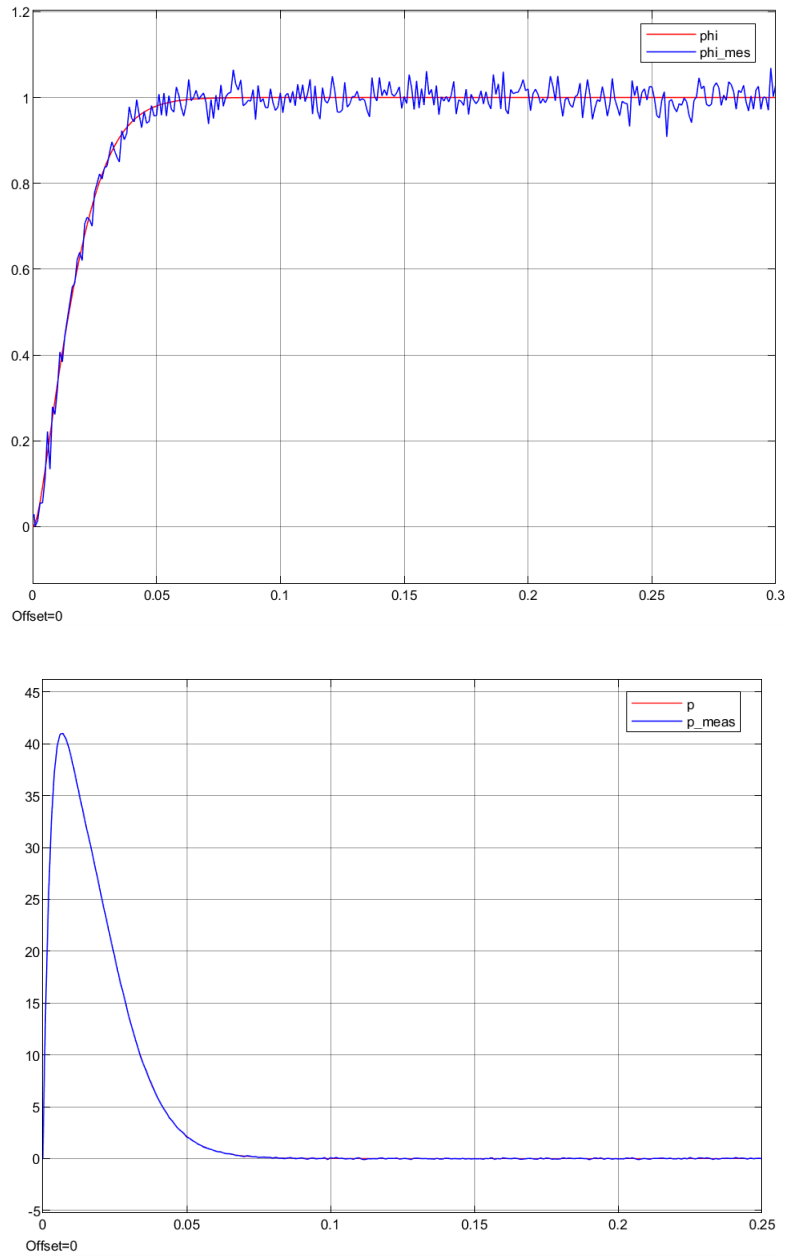


Figure 9 True versus measured roll angle (ϕ) and roll rate (Kalman Filter Design and Tuning)

7.2 Kalman Filter

Three independent steady-state discrete Kalman filters were designed — one per axis — and implemented directly as recursive predict-correct loops in Simulink, rather than as offline MATLAB scripts, since later phases require the filters to run live, in-loop, alongside the coupled dynamics. Measurement noise covariance (R) was set directly from the measured Phase 5 sensor noise, not assumed. A systematic sweep of the process-noise covariance's rate-channel entry (Q_{22}) was run per axis to correct an initial deficiency: every axis's own rate estimate initially performed worse than the raw sensor measurement, due to filter lag on the sharp rate transient.

Table 5 Per-axis raw rate signal-to-noise ratio, optimal process-noise tuning, and resulting rate-filter improvement.

Axis	Raw rate SNR (% of peak)	Optimal Q_{22}	Rate-filter improvement
Yaw	1.20% (worst)	0.8	+13.4% (best result)
Roll	1.04%	0.8	+11.7%
Pitch	0.62% (best)	1.0	~0% (no meaningful improvement)

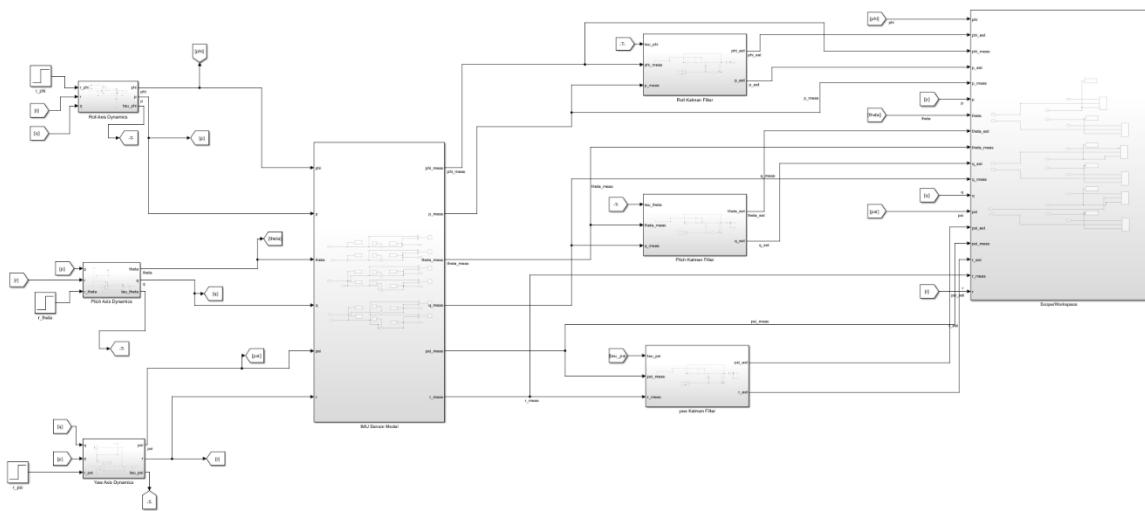


Figure 10 Kalman Filter Block

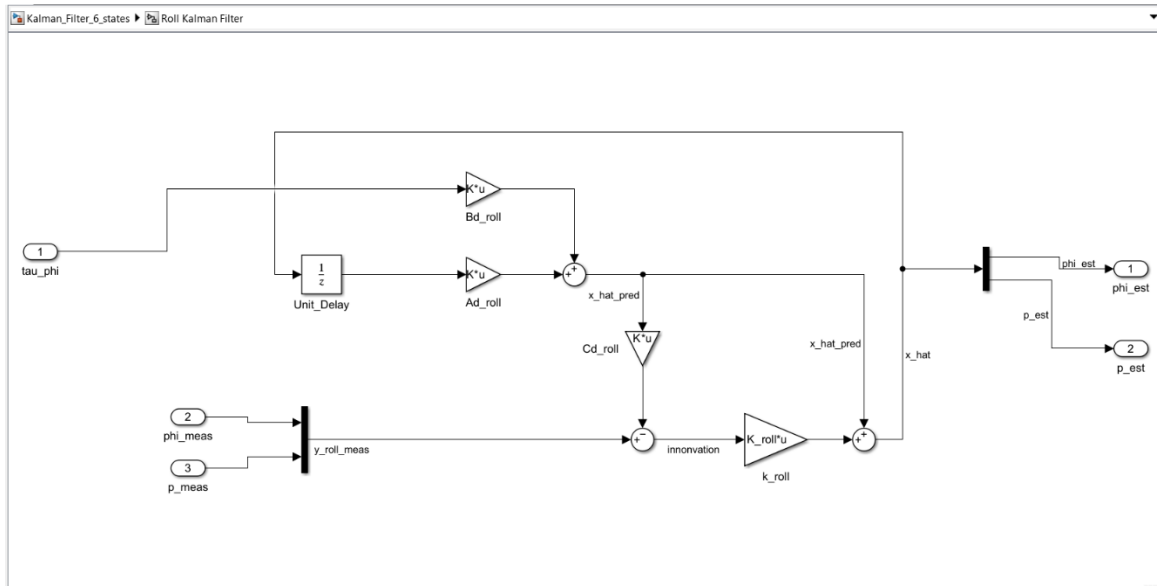


Figure 11 Kalman filter Simulink implementation

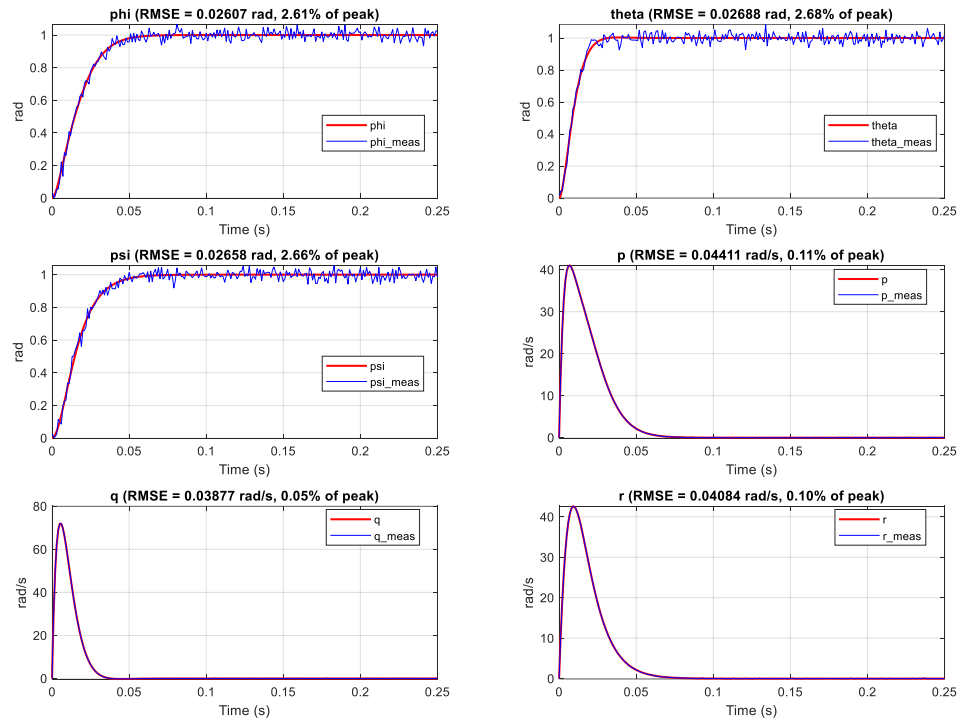


Figure 12 True, measured, and Kalman-estimated angle

This pattern — filter improvement inversely correlated with each axis's raw signal-to-noise ratio — makes physical sense: the cleaner the raw signal already is, the less room any filter has to improve it. Pitch's excellent raw rate SNR left its Kalman filter unable to demonstrate any meaningful benefit; its steady-state gain converged to approximately $K \approx 1$, meaning its estimated rate is, in practice, close to the raw unfiltered measurement. This single fact — established here in Phase 5 — is the direct root cause traced through every subsequent phase's pitch-specific findings.

8 Phase 6 — Closed-Loop Estimate Feedback and Disturbance Rejection

8.1 Closing the Loop on Estimated State

Every prior test used the plant's true state as controller feedback — a non-physical assumption, since a real flight computer only ever has access to sensor-derived estimates. Feedback was rerouted from true state to each axis's own Kalman-filtered estimate, for all three axes simultaneously, matching a real onboard architecture.

Axis	Settling-time penalty (true → estimate feedback)	New overshoot
Roll	$\sim 3.5\times$ worse	Small, previously 0%
Pitch	$\sim 2.6\times$ worse	Small, previously 0%
Yaw	Comparable to baseline, fastest recovery	Small, previously 0%

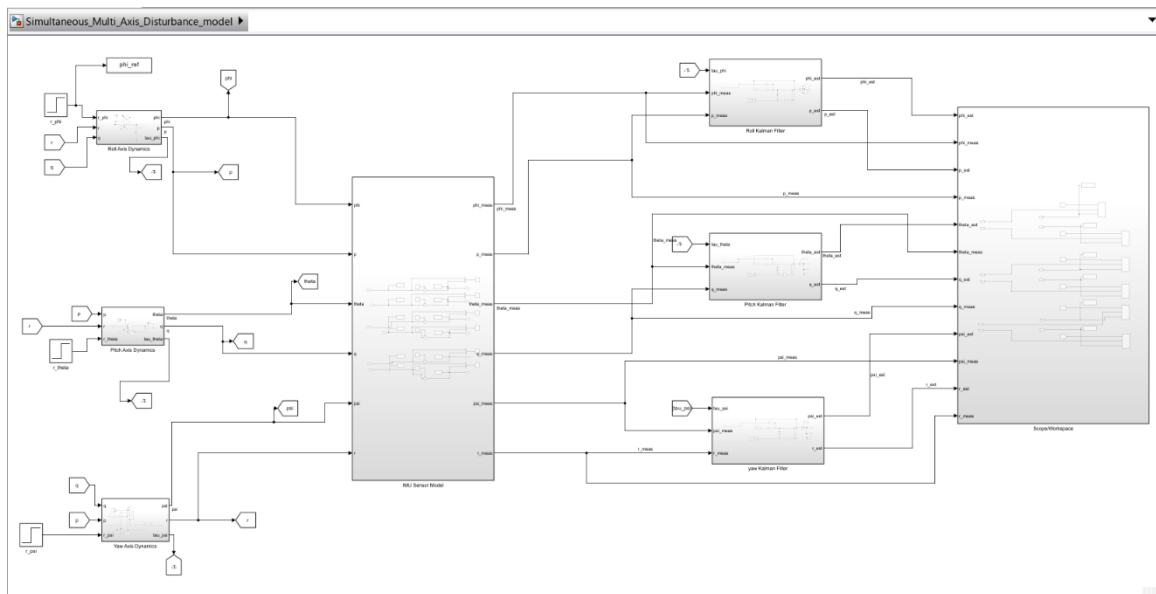


Figure 13 Full closed-loop architecture.

Under coupled, multi-axis commands, roll and yaw remained robust even at the aggressive 57.3° case (0% and 0.78% overshoot respectively). Pitch failed dramatically — 19.7% overshoot at 10° , 102.65% overshoot at 57.3° , never reaching settled tracking — confirmed as a compounding of Phase 4's coupling-sensitivity finding and Phase 5's ineffective rate filter: pitch is simultaneously the axis most exposed to coupling disturbance and the axis whose controller has the least-filtered information about it.

8.2 Disturbance Rejection

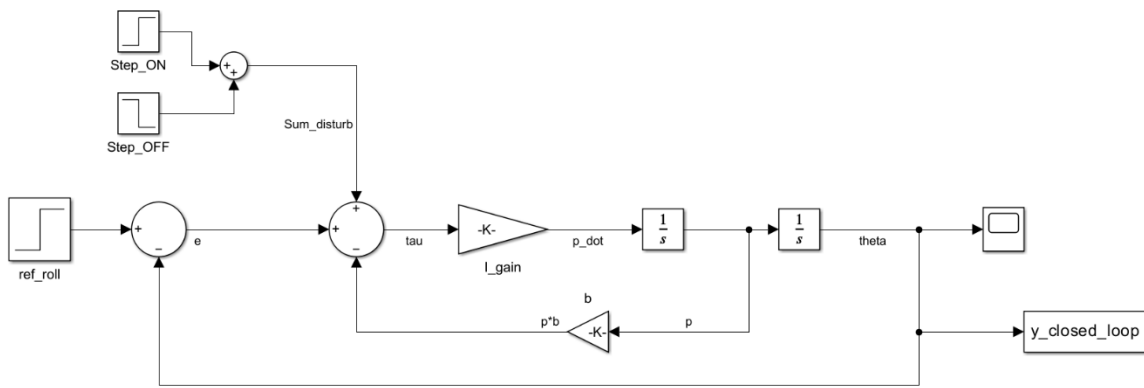


Figure 14 Disturbance pulse construction

A calibrated torque pulse (approximately 15–20% of each axis's own peak control torque) was injected, and rejection was compared under true-state versus estimate feedback.

Table 6 Peak deviation and recovery behaviour per axis under true-state versus estimate feedback.

Axis	Peak deviation (true → estimate)	Recovery behavior (estimate feedback)
Roll	4.03% → 6.63%	Recovers cleanly, 65 ms (~2.7× slower than true-state baseline)
Yaw	0.18% → 1.20%	Recovers cleanly, 10 ms — fastest recovery of any estimate-feedback test

Pitch	0.10% $\rightarrow$ 1.60% (16 $\times$ larger transient)	Never fully settles — persistent 2.49% RMS chatter around reference
-------	--	---

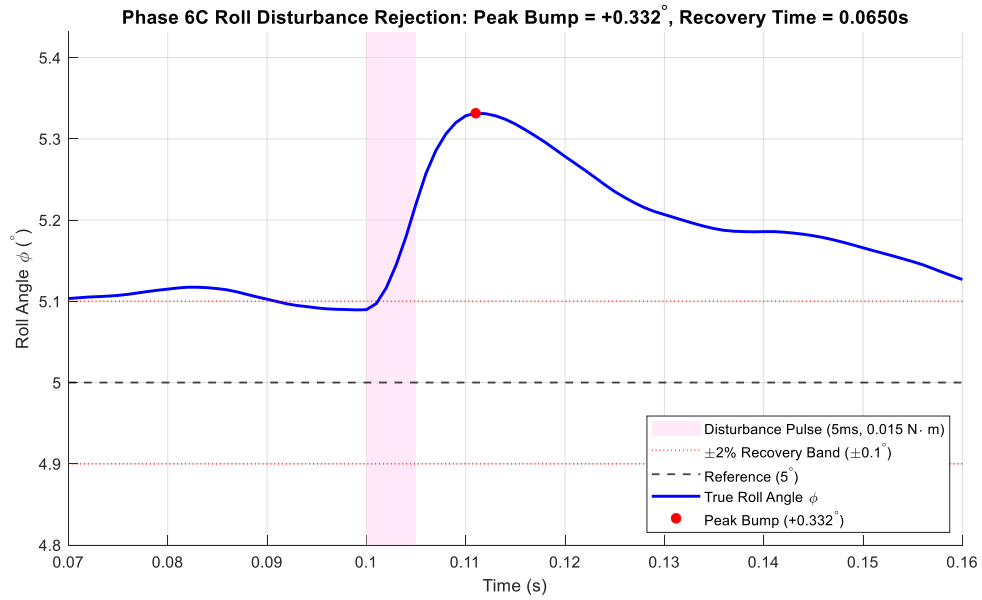


Figure 15 Roll disturbance rejection

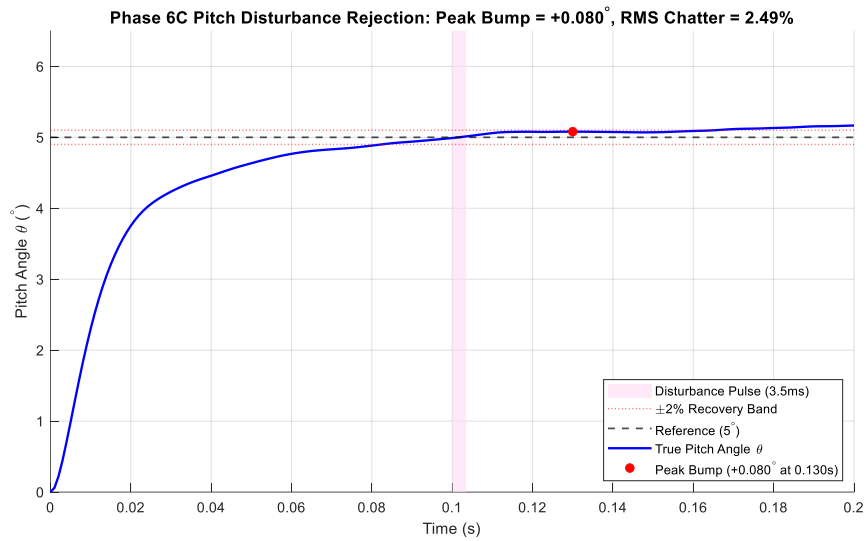


Figure 16 Pitch disturbance response

Pitch's failure mode is qualitatively distinct from roll's and yaw's: rather than a larger-but-bounded transient, it settles into low-amplitude, persistent oscillation that never converges within the test window — a direct consequence of its rate estimate providing essentially no noise attenuation.

8.3 Severity Sweep (1×–8× Baseline Disturbance)

Table 7 Severity sweep results: steady-state RMS chatter at 1× and 8× baseline disturbance, and the first crossing into unstable chatter.

Axis	1× RMS chatter	8× RMS chatter	First crossing into “unstable chatter”
Yaw	0.37%	0.52%	Never — settles cleanly throughout
Roll	2.40%	7.45%	8×
Pitch	2.29%	13.99%	4×

No axis diverged even at 8× the baseline disturbance in isolation — all remained bounded, confirming pitch's issue is one of steady-state precision, not closed-loop instability, at this severity range.

8.4 Simultaneous Multi-Axis Disturbance on the Coupled Model

Table 8 Simultaneous multi-axis disturbance results on the coupled model at 10° and 57.3°.

Axis	10° + 1× disturbance (RMS chatter)	57.3° + 1× disturbance (RMS chatter)
Roll	0.85% (settles cleanly)	0.33% (settles cleanly)
Pitch	15.83% (unstable chatter)	73.72% (unstable chatter)
Yaw	0.41% (settles cleanly)	0.28% (settles cleanly)

Pitch's chatter at just 10° with a single 1× disturbance (15.83%) already exceeds its isolated 8×-severity result (13.99%) — clear evidence that pitch's two vulnerabilities (coupling sensitivity and filter deficiency) compound multiplicatively rather than additively once both stressors are present simultaneously. Roll and yaw showed no such compounding, remaining excellent even under the combined worst case tested.

9 Phase 7 — Performance Evaluation and Operating Envelope

An acceptance-criteria framework was defined: overshoot $\leq 25\%$ acceptable, 5–20% degrading, above 20% unacceptable; RMS chatter $\leq 2\%$ acceptable, 2–5% degrading, above 5% unacceptable; and a hard boundedness limit at 3× the commanded reference, beyond which a result is classified as diverged rather than merely unacceptable.

Table 9 Acceptance criteria thresholds applied throughout Phases 7 and 8.

Metric	Acceptable	Degrading	Unacceptable
Overshoot	$\leq 5\%$	5–20%	$> 20\%$
Steady-state RMS chatter	$\leq 2\%$	2–5%	$> 5\%$
Settling	Reaches band within test window	—	Never settles
Boundedness	—	—	Exceeds 3× reference (classified as diverged)

An isolation test — the coupled model with estimate feedback, but with the Phase 6 disturbance removed entirely — was run at 5°, 10°, and 57.3° to determine whether pitch's failure was driven primarily by coupling, by disturbance, or by both together.

Table 10 Pitch performance, coupled-only versus coupled-plus-disturbance, at each tested commanded angle.

Angle	Pitch, coupled only	Pitch, coupled + disturbance
5°	10.38% overshoot, 7.93% chatter, never settles	10.38% overshoot, 9.37% chatter, never settles
10°	19.70% overshoot, 15.11% chatter, never settles	19.70% overshoot, 15.83% chatter, never settles
57.3°	102.68% overshoot, 73.59% chatter, never settles	102.68% overshoot, 73.72% chatter, never settles

Removing the disturbance changed pitch's overshoot by exactly zero and its chatter by only a fraction of a percentage point at every angle tested. This isolates coupling as the dominant driver of pitch's performance failure within the disturbance severity used in this phase (the Phase 6A 1× baseline) — not necessarily across all disturbance levels; Phase 8 subsequently shows severity does matter more at small angles when pushed to substantially larger multiples. No acceptable pitch operating point was identified within the tested simultaneous-maneuver range of 5–57.3°, whether or not a disturbance was present.

Roll and yaw remained acceptable across this entire range, with or without disturbance, establishing that the architecture's limitation is specific to pitch rather than general.

10 Phase 8 — Stress Testing to the Failure Boundary

A combined two-dimensional sweep — 8 commanded angles (5°, 10°, 20°, 30°, 57.3°, 90°, 120°, 180°) crossed with 7 disturbance severities (1× through 64× the Phase 6A baseline), 56 total conditions — was run on the coupled, estimate-feedback model to locate the precise boundary between bounded-but-unacceptable performance and genuine divergence (state exceeding 3× the commanded reference).

10.1 Pitch's Divergence Boundary Is Angle-Driven

Across every disturbance severity from $1\times$ to $64\times$, pitch remained bounded — degraded, but not diverged — for every commanded angle from 5° through 120° . At exactly 180° , pitch diverged at every severity level tested, with overshoot ($\approx 210\%$) and chatter ($\approx 150\text{--}153\%$) essentially unchanged across the full $64\times$ severity range. Within this simulation, attitude magnitude — not disturbance severity — was the variable most associated with the transition to genuine divergence.

This finding carries an important caveat: 180° commands a fully inverted attitude, well beyond where the small-angle approximation underlying the Euler coupling model ($\sin \theta \approx \theta$) remains valid. The divergence is real within the simulation and the linearized model's own domain, but it cannot be interpreted as a physical failure angle for the actual vehicle — it reflects the model's own validity limit as much as it reflects the architecture's behavior.

10.2 Severity Matters More at Small Angles

At 5° , pitch's chatter grew roughly 13-fold across the tested severity range (9.37% at $1\times$ to 119.52% at $64\times$). At 57.3° , the same severity range moved chatter by less than 15% relative (73.7% to 83.2%). Disturbance severity dominates pitch's degradation at small angles; coupling dominates and effectively saturates the failure at large angles, leaving comparatively little room for severity to worsen the outcome further.

10.3 Roll's Single Failure Point

Roll remained acceptable or mildly degrading across the entire 56-condition grid except one corner: 5° combined with $64\times$ disturbance severity, where it genuinely diverged (297.89% overshoot). This is consistent with Phase 4's finding that roll's coupling coefficient is negative (damping-like) and that coupling strength scales with commanded angle — at small angles, roll receives very little of this protective effect, leaving it exposed only at extreme disturbance severity. Notably, roll's robustness improved steadily as angle increased at every severity level — the opposite trend from pitch.

10.4 Yaw's Complete Robustness

Across all 56 tested angle-severity combinations, yaw never once crossed into “unacceptable” or “diverged” — the most robust axis in the architecture by a clear margin, consistent with its best-in-class Kalman filter tuning established in Phase 5.

Table 11 Summary of divergence conditions identified across all 56 tested angle-severity combinations. Yaw does not appear — it did not diverge under any tested condition.

Axis	Angle	Severity	Overshoot	RMS Chatter
Pitch	180°	1×	209.85%	150.03%
Pitch	180°	2×	209.85%	150.07%
Pitch	180°	4×	209.85%	150.15%
Pitch	180°	8×	209.85%	150.31%
Pitch	180°	16×	209.85%	150.64%
Pitch	180°	32×	209.85%	151.32%
Pitch	180°	64×	209.85%	152.82%
Roll	5°	64×	297.89%	65.29%

Table 12 Combined verdict per axis across nominal, aggressive, and extreme operating regimes.

Axis	Nominal ($\leq 10^\circ$)	Aggressive (up to $\sim 90^\circ$)	Extreme (approaching model validity limits)
Roll	Acceptable	Acceptable (improves with angle)	Diverges only at small angle combined with 64× disturbance
Pitch	Unacceptable performance (bounded chatter, not instability)	Unacceptable performance (still bounded)	Genuine divergence beyond $\sim 120\text{--}180^\circ$, outside small-angle validity

Yaw	Acceptable	Acceptable	Acceptable — no failure found anywhere in the tested range
-----	------------	------------	--

11 Aerospace Relation

1. Attitude and rate control architectures of this type — P-control with Kalman-filtered feedback — are standard building blocks in real small-UAV and CubeSat attitude-control systems.
2. The observed decoupling between filter tuning quality and disturbance robustness demonstrates why real flight-software verification treats estimator tuning as a first-class design parameter, not an afterthought to controller design.
3. Nano-scale UAVs, like the Black Hornet class modeled here, exhibit unusually large angular rates relative to their control torques — a scaling effect that becomes increasingly important as UAV designs shrink.
4. The demonstrated multiplicative interaction between coupling and estimator error is directly relevant to any coupled-axis vehicle (rockets, missiles, agile aircraft) where multiple rotational degrees of freedom are active simultaneously.
5. This architecture and its verification methodology form a direct prerequisite for Project 14's rocket attitude-control system, which will face an analogous coupled-dynamics, sensor-realistic estimation problem.

12 Key Engineering Conclusions

- A control architecture's overall soundness and a single component's tuning quality are separable findings — roll and yaw demonstrate the architecture itself is sound; pitch's limitation is precisely traceable to one Kalman filter's tuning, not the controller design, plant model, or shared sensor model.
- Disturbance rejection quality under estimate feedback tracks directly with each axis's Kalman filter rate-tuning quality (Phase 5), not with axis bandwidth, inertia, or any other structural property.
- Structural coupling sensitivity and estimator tuning deficiency compound multiplicatively, not additively, when both are present simultaneously — a materially more serious finding than either limitation in isolation would suggest.
- “Stable but unacceptable” and “genuinely unstable” are distinct failure modes with different engineering implications; across nearly the entire tested envelope, pitch's failure is the former — a precision problem, not a stability problem.
- The identified fix is targeted — a coupling-aware (multi-axis or Extended) Kalman filter for pitch specifically — not a redesign of the controller or overall architecture.

13 Model Limitations and Assumptions Summary

- Inertia values are geometric approximations, not measurements of a real airframe.
- Aerodynamic damping was not measured; damping was set via theoretical critical-damping design.
- Rotor gyroscopic coupling (distinct from Euler rigid-body coupling) was identified as relevant but not modeled.
- Sensor noise levels are drawn from a representative MEMS IMU datasheet, not measured hardware; sensor bias/drift was excluded.
- All three Kalman filters are linear and single-axis; none incorporate the coupling dynamics into their internal model — this is the direct subject of the project's central finding.
- Results at attitude magnitudes beyond roughly 90–120° exceed the small-angle validity of the coupling model and should be read as simulation-domain findings, not physical predictions.

14 Future Work

- Design and evaluate a coupling-aware or Extended Kalman Filter specifically for the pitch channel and re-run the Phase 6–8 test suite to quantify the improvement.
- Model rotor gyroscopic coupling explicitly, given the vehicle's coaxial contra-rotating rotor configuration, and quantify its contribution alongside the already-modeled Euler coupling.
- Extend the plant model beyond small-angle assumptions to properly characterize behavior at large attitude excursions, rather than treating them as outside the model's scope.
- Incorporate sensor bias/drift into the IMU model for a more complete estimator stress test.
- Carry this architecture and verification methodology forward into Project 14 (Rocket Attitude Control), where an analogous coupled, sensor-realistic estimation problem is expected.

15 Appendix

15.1 MATLAB Code

```
% Roll-axis b-sweep – Project 13
clear; clc;

modelName = 'roll'; % <-- replace with your .slx filename (no
extension)
```

```

open_system(modelName);

% Block path to your damping gain block (the "b" triangle gain in
your closed-loop subsystem)
% Right-click the block -> "Copy path to clipboard" gets you this
exact string
bBlockPath = [modelName '/b/']; % <-- replace with your actual
path

% Values to sweep

bValues = [0.001, 0.005, 0.0154, 0.03, 0.1, 1];

% Table to collect results
results = table('Size',[length(bValues) 5], ...
    'VariableTypes',
    {'double','double','double','double','double'}, ...
    'VariableNames',
    {'b','RiseTime','SettlingTime','Overshoot','Peak'});

for i = 1:length(bValues)
    b_current = bValues(i);

    % Push this b value into the gain block
    set_param(bBlockPath, 'Gain', num2str(b_current));

    % Run the simulation
    simOut = sim(modelName);

    y = simOut.y_closed_loop.Data;
    t = simOut.y_closed_loop.Time;

    info = stepinfo(y, t);

    results.b(i) = b_current;
    results.RiseTime(i) = info.RiseTime;
    results.SettlingTime(i) = info.SettlingTime;
    results.Overshoot(i) = info.Overshoot;
    results.Peak(i) = info.Peak;

    fprintf('b = %.4f done - RiseTime: %.3f, Settling: %.3f,
Overshoot: %.2f%%\n', ...
        b_current, info.RiseTime, info.SettlingTime,
        info.Overshoot);

```

```

end

disp(results)

% Quick visual: overshoot and settling time vs b
figure;
subplot(2,1,1)
semilogx(results.b, results.Overshoot, '-o', 'LineWidth', 2)
xlabel('b'); ylabel('Overshoot (%)'); grid on; title('Overshoot
vs Damping')

subplot(2,1,2)
semilogx(results.b, results.SettlingTime, '-o', 'LineWidth', 2)
xlabel('b'); ylabel('Settling Time (s)'); grid on;
title('Settling Time vs Damping')

how i am using this code

%% Project 13: Sanity Check Automation (Roll, Pitch, Yaw Isolated
5-deg Steps)
clear; close all;

modelName = 'Coupling'; % <-- UPDATE THIS to your exact Simulink
model name

if ~bdIsLoaded(modelName)
    load_system(modelName);
end

% Set simulation duration
set_param(modelName, 'StopTime', '0.2');

% 5 degrees step amplitude in radians
cmd_5deg = deg2rad(57.3); % ~0.087266 rad

% Table to collect sanity check performance metrics
tests = {'Roll_Only', 'Pitch_Only', 'Yaw_Only'};
metricsTable = table('Size', [3, 7], ...
    'VariableTypes', {'string', 'double', 'double', 'double',
'double', 'double', 'double'}, ...
    'VariableNames', {'Test_Axis', 'RiseTime_s',
'SettlingTime_s', 'SettlingMin', 'SettlingMax', 'Overshoot_pct',
'Peak'});

```

```

figure('Name', 'Sanity Check Isolated 57.3-deg Step Responses',
'Position', [100, 100, 1000, 600]);

for i = 1:length(tests)
    testName = tests{i};

    % Configure Step inputs
    switch testName
        case 'Roll_Only'
            set_param([modelName '/r_phi'], 'After',
num2str(cmd_5deg), 'Time', '0');
            set_param([modelName '/r_theta'], 'After', '0',
'Time', '0');
            set_param([modelName '/r_psi'], 'After', '0',
'Time', '0');
            targetSignal = 'phi';

            case 'Pitch_Only'
                set_param([modelName '/r_phi'], 'After', '0',
'Time', '0');
                set_param([modelName '/r_theta'], 'After',
num2str(cmd_5deg), 'Time', '0');
                set_param([modelName '/r_psi'], 'After', '0',
'Time', '0');
                targetSignal = 'theta';

            case 'Yaw_Only'
                set_param([modelName '/r_phi'], 'After', '0',
'Time', '0');
                set_param([modelName '/r_theta'], 'After', '0',
'Time', '0');
                set_param([modelName '/r_psi'], 'After',
num2str(cmd_5deg), 'Time', '0');
                targetSignal = 'psi';
        end

    % Run simulation
    simOut = sim(modelName);

    % Extract primary response
    sigData = simOut.(targetSignal).Data;
    t = simOut.(targetSignal).Time;

    % Compute step performance characteristics
    info = stepinfo(sigData, t, cmd_5deg);

```

```

metricsTable.Test_Axis(i)      = testName;
metricsTable.RiseTime_s(i)    = info.RiseTime;
metricsTable.SettlingTime_s(i) = info.SettlingTime;
metricsTable.SettlingMin(i)   = info.SettlingMin;
metricsTable.SettlingMax(i)   = info.SettlingMax;
metricsTable.Overshoot_pct(i) = info.Overshoot;
metricsTable.Peak(i)          = info.Peak;

% Plot 3-axis tracking
subplot(3, 1, i);
plot(t, rad2deg(simOut.phi.Data), 'r', 'LineWidth', 1.5);
hold on;
plot(t, rad2deg(simOut.theta.Data), 'g', 'LineWidth', 1.5);
plot(t, rad2deg(simOut.psi.Data), 'b', 'LineWidth', 1.5);
yline(5, '--k', 'Command (57.3 deg)');
grid on;
ylabel('Angle (deg)');
title([strrep(testName, '_', ' ') ' Step Response']);
legend('\phi (Roll)', '\theta (Pitch)', '\psi (Yaw)',
'Location', 'SouthEast');
end

xlabel('Time (s)');

fprintf('\n===== SANITY CHECK METRICS TABLE (57.3-
DEG STEPS) =====\n');
disp(metricsTable);

%% Project 13: Simultaneous 3-Axis Multi-Amplitude Coupled Test

modelName = 'Coupling'; % Replace with your exact model name
if ~bdIsLoaded(modelName), load_system(modelName); end
set_param(modelName, 'StopTime', '0.2');
amplitudes_deg = [5, 10, 57.3];
amplitudes_rad = deg2rad(amplitudes_deg);
figure('Name', 'Simultaneous 3-Axis Step Response Across
Amplitudes', 'Position', [100, 100, 1100, 700]);
simResults = table();
for k = 1:length(amplitudes_rad)
    amp = amplitudes_rad(k);
    amp_deg = amplitudes_deg(k);

    % Apply simultaneous step commands to ALL three axes at t = 0
    set_param([modelName '/r_phi'], 'After', num2str(amp),
'Time', '0');

```

```

    set_param([modelName '/r_theta'], 'After', num2str(amp),
'Time', '0');
    set_param([modelName '/r_psi'], 'After', num2str(amp),
'Time', '0');

    simOut = sim(modelName);

    t = simOut.phi.Time;
    phi = simOut.phi.Data;
    theta = simOut.theta.Data;
    psi = simOut.psi.Data;

    info_phi = stepinfo(phi, t, amp);
    info_theta = stepinfo(theta, t, amp);
    info_psi = stepinfo(psi, t, amp);

    % Store table row
    row = table(amp_deg, ...
        info_phi.RiseTime, info_phi.SettlingTime,
info_phi.Overshoot, ...
        info_theta.RiseTime, info_theta.SettlingTime,
info_theta.Overshoot, ...
        info_psi.RiseTime, info_psi.SettlingTime,
info_psi.Overshoot, ...
        'VariableNames', {'Amp_deg', ...
        'Roll_tr', 'Roll_ts', 'Roll_OS', ...
        'Pitch_tr', 'Pitch_ts', 'Pitch_OS', ...
        'Yaw_tr', 'Yaw_ts', 'Yaw_OS'});

    simResults = [simResults; row];

    % Plot response curves
    subplot(3, 1, k);
    plot(t, rad2deg(phi), 'r', 'LineWidth', 1.5); hold on;
    plot(t, rad2deg(theta), 'g', 'LineWidth', 1.5);
    plot(t, rad2deg(psi), 'b', 'LineWidth', 1.5);
    yline(amp_deg, '--k', sprintf('Command (%.1f deg)',
amp_deg));
    grid on;
    ylabel('Angle (deg)');
    title(sprintf('Simultaneous %.1f^\circ Step (Roll, Pitch,
Yaw Active)', amp_deg));
    legend('\phi (Roll)', '\theta (Pitch)', '\psi (Yaw)',
'Location', 'SouthEast');
end
xlabel('Time (s)');

```

```

fprintf('\n===== SIMULTANEOUS 3-AXIS PERFORMANCE
TABLE =====\n');
disp(simResults);

% Phase 5B – RMSE Analysis: True vs. Measured States (To
Workspace blocks)
clear; clc;

modelName = 'Sensor_Model'; % <-- replace with your actual .slx
filename
open_system(modelName);

sim(modelName); % To Workspace blocks write directly into the
base workspace

% Match these to the EXACT "Variable name" you set in each To
Workspace block
trueVarNames = {'phi', 'theta', 'psi', 'p', 'q', 'r'};
measVarNames = {'phi_meas', 'theta_meas', 'psi_meas', 'p_meas',
'q_meas', 'r_meas'};
labels       = {'phi', 'theta', 'psi', 'p', 'q', 'r'};
units        = {'rad', 'rad', 'rad', 'rad/s', 'rad/s', 'rad/s'};

results = table('Size', [6 5], ...
    'VariableTypes',
    {'string', 'double', 'double', 'double', 'double'}, ...
    'VariableNames',
    {'Signal', 'RMSE', 'PeakTrueValue', 'RMSE_pct_of_peak', 'NoiseSTD'});

figure('Name', 'True vs Measured – All Channels', 'Position', [100
100 1200 800]);

for i = 1:6
    trueStruct = eval(trueVarNames{i}); % pulls the variable by
name from base workspace
    measStruct = eval(measVarNames{i});

    t_true = trueStruct.Time;
    y_true = trueStruct.Data;
    t_meas = measStruct.Time;
    y_meas = measStruct.Data;

    % Interpolate measured onto true's time vector in case sample
times differ

```

```

    y_meas_interp = interp1(t_meas, y_meas, t_true, 'linear',
'extrap');

    err = y_true - y_meas_interp;
    rmse = sqrt(mean(err.^2));
    peakVal = max(abs(y_true));
    rmse_pct = 100 * rmse / peakVal;
    noiseSTD = std(err);

    results.Signal(i)          = labels{i};
    results.RMSE(i)             = rmse;
    results.PeakTrueValue(i)    = peakVal;
    results.RMSE_pct_of_peak(i) = rmse_pct;
    results.NoiseSTD(i)         = noiseSTD;

    subplot(3,2,i);
    plot(t_true, y_true, 'r', 'LineWidth', 1.5); hold on;
    plot(t_true, y_meas_interp, 'b');
    grid on;
    title(sprintf('%s (RMSE = %.4g %s, %.2f%% of peak)',
labels{i}, rmse, units{i}, rmse_pct));
    legend(labels{i}, [labels{i} '\_meas'], 'Location', 'best');
    xlabel('Time (s)'); ylabel(units{i});
end

fprintf('\n===== RMSE SUMMARY – TRUE vs MEASURED
===== \n');
disp(results)

```

```

%% Pitch Kalman Filter Design
I_pitch = 2.76e-5;
b_crit_pitch = 0.0105;
Ts = 0.001;

A_p = [0 1; 0 -b_crit_pitch/I_pitch];
B_p = [0; 1/I_pitch];
C_p = [1 0; 0 1];
D_p = [0; 0];

sys_pitch_d = c2d(ss(A_p,B_p,C_p,D_p), Ts);

Ad_pitch = sys_pitch_d.A;
Cd_pitch = sys_pitch_d.C;

```

```

Bd_pitch = sys_pitch_d.B;

R_pitch = diag([0.026881^2, 0.038771^2]); % theta, q noise std
from your Phase 5B table
Q_pitch = diag([0.01, 0.01]) ; % same starting point
as roll

[~, K_pitch, ~] = dlqe(Ad_pitch, eye(2), Cd_pitch, Q_pitch,
R_pitch);

A_cl_pitch = (eye(2) - K_pitch*Cd_pitch) * Ad_pitch;
disp('K_pitch:'); disp(K_pitch)
disp('Pole magnitudes:'); disp(abs(eig(A_cl_pitch)))

%% Kalman Filter for roll and roll rate

b_crtitital_roll = 0.0154;
I_roll = 5.94e-5;

A = [0 1 ; 0 -b_crtitital_roll/I_roll] ;

B = [0 ; 1/I_roll];

C = [1 0;0 1];

D = [0;0];

sys_roll = ss(A,B,C,D);

% Continuous to Discrete Time

Ts = 0.001;

sys_roll_d = c2d(sys_roll,Ts);

Ad_roll = sys_roll_d.A;
Bd_roll = sys_roll_d.B;
Cd_roll = sys_roll_d.C;
Dd = sys_roll_d.D;

Q = diag([0.01, 0.8]);

R = diag([0.0268^2, 0.0436^2]);

```

```

[~,K_roll,~] = dlqe(Ad_roll,eye(2),Cd_roll,Q,R);

A_cl = (eye(2) - K_roll*Cd_roll) * Ad_roll;
disp('K_roll:'); disp(K_roll)
disp('Pole magnitudes:'); disp(abs(eig(A_cl)))

%% Yaw Kalman Filter Design
I_yaw = 8.70e-5;
b_crit_yaw = 0.0187;
Ts = 0.001;

A_y = [0 1; 0 -b_crit_yaw/I_yaw];
B_y = [0; 1/I_yaw];
C_y = [1 0; 0 1];
D_y = [0; 0];

sys_yaw_d = c2d(ss(A_y,B_y,C_y,D_y), Ts);
Ad_yaw = sys_yaw_d.A;
Cd_yaw = sys_yaw_d.C;
Bd_yaw = sys_yaw_d.B;

R_yaw = diag([0.026575^2, 0.040843^2]); % psi, r noise std from
your Phase 5B table
Q_yaw = diag([0.01, 0.8])

[~, K_yaw, ~] = dlqe(Ad_yaw, eye(2), Cd_yaw, Q_yaw, R_yaw);
A_cl_yaw = (eye(2) - K_yaw*Cd_yaw) * Ad_yaw;
disp('K_yaw:'); disp(K_yaw)
disp('Pole magnitudes:'); disp(abs(eig(A_cl_yaw)))

%% Phase 5D – Closed-Loop Performance: True State vs Estimated
State vs Reference

modelName = 'Kalman_Filter_6_states';
open_system(modelName);
sim(modelName);

trueVarNames = {'phi', 'theta', 'psi', 'p', 'q', 'r'};
estVarNames = {'phi_est', 'theta_est', 'psi_est', 'p_est',
'q_est', 'r_est'};
labels = {'phi', 'theta', 'psi', 'p', 'q', 'r'};
units = {'rad', 'rad', 'rad', 'rad/s', 'rad/s', 'rad/s'};

```

```

commanded    = [true, false, false, true, false, false]; % phi
& p are roll's angle/rate
refAmplitude = [deg2rad(5), 0, 0, NaN, 0, 0]; % rate axes don't
have a "reference angle" – see note below

results = table('Size', [6 6], ...
    'VariableTypes',
    {'string','double','double','double','double','double'}, ...
    'VariableNames',
    {'Signal','Commanded','RiseTime_or_NaN','SettlingTime_or_NaN',
    ...
    'Overshoot_or_NaN','RMS_chatter_or_SSError'}));

figure('Name','True vs Estimated State – Closed Loop (Phase 5D,
fixed)','Position',[100 100 1300 850]);

for i = 1:6
    trueData = evalin('base', trueVarNames{i});
    estData  = evalin('base', estVarNames{i});

    t = trueData.Time;
    y_true = trueData.Data(:);
    y_est_interp = interp1(estData.Time, estData.Data(:), t,
    'linear', 'extrap');

    results.Signal(i) = labels{i};
    results.Commanded(i) = commanded(i);

    if commanded(i) && ~isnan(refAmplitude(i)) && refAmplitude(i)
    ~= 0
        % Genuinely commanded axis with a real target angle –
        real stepinfo
        info = stepinfo(y_true, t, refAmplitude(i));
        results.RiseTime_or_NaN(i)      = info.RiseTime;
        results.SettlingTime_or_NaN(i) = info.SettlingTime;
        results.Overshoot_or_NaN(i)    = info.Overshoot;
        results.RMS_chatter_or_SSError(i) = abs(refAmplitude(i) -
        y_true(end));
    else

        results.RiseTime_or_NaN(i)      = NaN;
        results.SettlingTime_or_NaN(i) = NaN;
        results.Overshoot_or_NaN(i)    = NaN;

```

```

        results.RMS_chatter_or_SSError(i) =
sqrt(mean(y_true.^2));
    end

    subplot(3,2,i);
    plot(t, y_true, 'k', 'LineWidth', 1.6); hold on;
    plot(t, y_est_interp, 'r--', 'LineWidth', 1.1);
    if commanded(i) && ~isnan(refAmplitude(i)) && refAmplitude(i)
~= 0
        yline(refAmplitude(i), ':b', 'Reference');
        ttl = sprintf('%s - commanded (RMSE true/est=%.4g)',
labels{i}, sqrt(mean((y_true-y_est_interp).^2)));
    else
        yline(0, ':b', 'Zero ref');
        ttl = sprintf('%s - uncommanded, RMS chatter=%.4g',
labels{i}, sqrt(mean(y_true.^2)));
    end
    grid on; title(ttl);
    legend(labels{i}, [labels{i} '\_est'], 'Location', 'best');
    xlabel('Time (s)'); ylabel(units{i});
end

fprintf('\n===== PHASE 5D CLOSED-LOOP PERFORMANCE
(fixed) =====\n');
disp(results)

%% Phase 6B – Pitch Disturbance Rejection Analysis (True-State
Feedback)
clear; clc;

% 1. Simulation Setup
modelName = 'Single_Axis_Pitch_Disturbance';
open_system(modelName);
sim(modelName);

data = y_closed_loop;
t = data.Time;
y = data.Data(:);

% 3. Disturbance & Reference Parameters
refValue = deg2rad(5);           % same 5° commanded angle
disturbanceStart = 0.07;         % pitch's onset, not roll's 0.1
disturbanceEnd = 0.0735;         % pitch's pulse end

% 4. Isolate the Post-Disturbance Window (t >= 0.1s)
idx_post = t >= disturbanceStart;

```

```

t_post    = t(idx_post);
y_post    = y(idx_post);

% 5. Peak Deviation (Maximum bump caused by torque pulse)
deviation    = y_post - refValue;
[maxDev, maxIdx] = max(abs(deviation));
peakTime     = t_post(maxIdx);
peakAngle    = y_post(maxIdx);

% 6. Recovery Time (2% Settling Band around Reference)
band         = 0.02 * refValue;    % +/- 2% of 5 deg (~0.1 deg)
recovered    = false;
recoveryTime = NaN;

for k = maxIdx:length(y_post)
    % Verify the trajectory enters and remains within the band
    indefinitely
        if all(abs(y_post(k:end) - refValue) <= band)
            recoveryTime = t_post(k) - disturbanceStart;
            recovered    = true;
            break;
        end
    end
end

% 7. Display Key Metrics
fprintf('\n===== PHASE 6B – DISTURBANCE REJECTION
=====');
fprintf('Reference Target Angle           : %.5f rad (%.2f deg)\n',
refValue, rad2deg(refValue));
fprintf('Peak Deviation from Target         : %.5f rad (%.3f deg)\n',
maxDev, rad2deg(maxDev));
fprintf('Peak Deviation Percentage           : %.2f %% of
reference\n', (maxDev / refValue) * 100);
fprintf('Time to Reach Peak Deviation       : %.4f s (after pulse
onset)\n', peakTime - disturbanceStart);

if recovered
    fprintf('Recovery Time (to 2%% band)       : %.4f s (after
pulse onset)\n', recoveryTime);
    fprintf('Total Settling Timestamp           : %.4f s\n',
disturbanceStart + recoveryTime);
else
    fprintf('Recovery Time                       : System did NOT
settle within 2%% band during simulation.\n');
end
end

```

```

fprintf('=====
=====\\n');

figure('Name','Phase 6B: Pitch Disturbance Zoomed
(Degrees)','Position',[150 150 950 520]);

% Convert to degrees
y_deg      = rad2deg(y);
refValue_deg = rad2deg(refValue);
band_deg    = rad2deg(band);
maxDev_deg  = rad2deg(maxDev);
peakAngle_deg = rad2deg(peakAngle);

% Define tight zoom window around the pitch pulse (0.07s -
0.0735s)
t_zoom_start = 0.055;
t_zoom_end   = 0.110;
idx_zoom     = (t >= t_zoom_start) & (t <= t_zoom_end);

% 1. Disturbance pulse shaded box (3.5 ms pulse)
patch([disturbanceStart disturbanceEnd disturbanceEnd
disturbanceStart], ...
      [4.85 4.85 5.15 5.15], [1 0.85 0.95], 'EdgeColor', 'none',
      ...
      'FaceAlpha', 0.6, 'DisplayName', 'Disturbance Pulse
(3.5ms)');
hold on;

% 2.  $\pm 2\%$  Recovery Band
yline(refValue_deg + band_deg, 'r', 'LineWidth', 1.1,
'DisplayName', '\\pm 2% Recovery Band');
yline(refValue_deg - band_deg, 'r', 'LineWidth', 1.1,
'HandleVisibility', 'off');

% 3. Reference line (5 deg)
yline(refValue_deg, 'k--', 'LineWidth', 1.2, 'DisplayName',
'Reference (5^\\circ)');

% 4. True Pitch Response
plot(t(idx_zoom), y_deg(idx_zoom), 'b', 'LineWidth', 2.0,
'DisplayName', 'True Pitch Angle \\theta');

% 5. Peak deviation point

```

```

plot(peakTime, peakAngle_deg, 'ro', 'MarkerFaceColor', 'r',
'MarkerSize', 6, ...
'DisplayName', sprintf('Peak Bump (+%.3f^\circ)',
maxDev_deg));

grid on;
xlim([t_zoom_start, t_zoom_end]);

% Y-axis headroom centered around the 5 deg setpoint
ylim([4.88, 5.03]);

xlabel('Time (s)', 'FontSize', 11);
ylabel('Pitch Angle \theta (^{\circ})', 'FontSize', 11);
title(sprintf('Pitch Disturbance Rejection (Zoomed): Peak Bump =
+%.3f^\circ, Recovery Time = %.4fs', ...
maxDev_deg, recoveryTime), 'FontSize', 12);

legend('Location', 'southeast', 'FontSize', 9);

%% Phase 6B – Roll Disturbance Rejection Analysis (True-State
Feedback)
clear; clc;

% 1. Simulation Setup
modelName = 'Single_Axis_Roll_Disturbance'; % <-- Put your
exact .slx file name here (without .slx)
open_system(modelName);
sim(modelName);

data = y_closed_loop;
t = data.Time;
y = data.Data(:);

% 3. Disturbance & Reference Parameters
refValue = deg2rad(5); % Commanded roll angle: 5 deg =
0.087266 rad
disturbanceStart = 0.100; % Disturbance pulse onset (s)
disturbanceEnd = 0.105; % Disturbance pulse end (s)

% 4. Isolate the Post-Disturbance Window (t >= 0.1s)
idx_post = t >= disturbanceStart;
t_post = t(idx_post);
y_post = y(idx_post);

% 5. Peak Deviation (Maximum bump caused by torque pulse)
deviation = y_post - refValue;

```

```

[maxDev, maxIdx] = max(abs(deviation));
peakTime        = t_post(maxIdx);
peakAngle       = y_post(maxIdx);

% 6. Recovery Time (2% Settling Band around Reference)
band            = 0.02 * refValue;    % +/- 2% of 5 deg (~0.1 deg)
recovered       = false;
recoveryTime    = NaN;

for k = maxIdx:length(y_post)
    % Verify the trajectory enters and remains within the band
    indefinitely
        if all(abs(y_post(k:end) - refValue) <= band)
            recoveryTime = t_post(k) - disturbanceStart;
            recovered    = true;
            break;
        end
    end
end

% 7. Display Key Metrics
fprintf('\n===== PHASE 6B – DISTURBANCE REJECTION
=====\\n');
fprintf('Reference Target Angle           : %.5f rad (0.2f deg)\\n',
refValue, rad2deg(refValue));
fprintf('Peak Deviation from Target       : %.5f rad (0.3f deg)\\n',
maxDev, rad2deg(maxDev));
fprintf('Peak Deviation Percentage          : 0.2f %% of
reference\\n', (maxDev / refValue) * 100);
fprintf('Time to Reach Peak Deviation       : 0.4f s (after pulse
onset)\\n', peakTime - disturbanceStart);

if recovered
    fprintf('Recovery Time (to 2%% band)      : 0.4f s (after
pulse onset)\\n', recoveryTime);
    fprintf('Total Settling Timestamp         : 0.4f s\\n',
disturbanceStart + recoveryTime);
else
    fprintf('Recovery Time                   : System did NOT
settle within 2%% band during simulation.\\n');
end
fprintf('=====\\n');

% 8. Single Clean Disturbance Rejection Plot (Degrees on Y-Axis)
figure('Name','Phase 6B: Disturbance Rejection
(Degrees)','Position',[150 150 950 520]);

```

```

% Convert radians to degrees for plotting
y_deg      = rad2deg(y);
refValue_deg = rad2deg(refValue);
band_deg    = rad2deg(band);
maxDev_deg  = rad2deg(maxDev);
peakAngle_deg = rad2deg(peakAngle);

% A. Subtle shaded region for the 5ms disturbance window
patch([disturbanceStart disturbanceEnd disturbanceEnd
disturbanceStart], ...
      [0 0 7 7], [1 0.85 0.95], 'EdgeColor', 'none', ...
      'FaceAlpha', 0.6, 'DisplayName', 'Disturbance Pulse
(5ms)');
hold on;

% B. Tolerance bounds (+/- 2% recovery band in degrees)
yline(refValue_deg + band_deg, ':r', 'LineWidth', 1.0,
'DisplayName', '\pm2% Recovery Band');
yline(refValue_deg - band_deg, ':r', 'LineWidth', 1.0,
'HandleVisibility', 'off');

% C. Reference setpoint (5 deg)
yline(refValue_deg, 'k--', 'LineWidth', 1.2, 'DisplayName',
'Reference (5^\circ)');

% D. True roll response in degrees
plot(t, y_deg, 'b', 'LineWidth', 1.8, 'DisplayName', 'True Roll
Angle \phi');

% E. Peak deviation point in degrees
plot(peakTime, peakAngle_deg, 'ro', 'MarkerFaceColor', 'r',
'MarkerSize', 6, ...
'DisplayName', sprintf('Peak Bump (+%.3f^\circ)',
maxDev_deg));

grid on;
xlim([0, 0.2]);
ylim([0, 6.5]); % Headroom up to 6.5 degrees

xlabel('Time (s)', 'FontSize', 11);
ylabel('Roll Angle \phi (^\circ)', 'FontSize', 11);
title(sprintf('Roll Disturbance Rejection: Peak Bump =
+%.3f^\circ, Recovery Time = %.4fs', ...
maxDev_deg, recoveryTime), 'FontSize', 12);

```

```

legend('Location', 'southeast', 'FontSize', 9);

%% Phase 6B – Yaw Disturbance Rejection Analysis (True-State
Feedback)
clear; clc;

% 1. Simulation Setup
modelName = 'Single_Axis_Yaw_Disturbance';
open_system(modelName);
sim(modelName);

data = y_closed_loop;
t = data.Time;
y = data.Data(:);

% 3. Disturbance & Reference Parameters
refValue = deg2rad(5);
disturbanceStart = 0.12;
disturbanceEnd = 0.1235;

% 4. Isolate the Post-Disturbance Window (t >= 0.1s)
idx_post = t >= disturbanceStart;
t_post = t(idx_post);
y_post = y(idx_post);

% 5. Peak Deviation (Maximum bump caused by torque pulse)
deviation = y_post - refValue;
[maxDev, maxIdx] = max(abs(deviation));
peakTime = t_post(maxIdx);
peakAngle = y_post(maxIdx);

% 6. Recovery Time (2% Settling Band around Reference)
band = 0.02 * refValue; % +/- 2% of 5 deg (~0.1 deg)
recovered = false;
recoveryTime = NaN;

for k = maxIdx:length(y_post)
    % Verify the trajectory enters and remains within the band
    indefinitely
        if all(abs(y_post(k:end) - refValue) <= band)
            recoveryTime = t_post(k) - disturbanceStart;
            recovered = true;
            break;
        end
    end
end

```

```

% 7. Display Key Metrics
fprintf('\n===== PHASE 6B – DISTURBANCE REJECTION
=====');
fprintf('Reference Target Angle           : %.5f rad (%.2f deg)\n',
refValue, rad2deg(refValue));
fprintf('Peak Deviation from Target       : %.5f rad (%.3f deg)\n',
maxDev, rad2deg(maxDev));
fprintf('Peak Deviation Percentage         : %.2f %% of
reference\n', (maxDev / refValue) * 100);
fprintf('Time to Reach Peak Deviation     : %.4f s (after pulse
onset)\n', peakTime - disturbanceStart);

if recovered
    fprintf('Recovery Time (to 2%% band)       : %.4f s (after
pulse onset)\n', recoveryTime);
    fprintf('Total Settling Timestamp         : %.4f s\n',
disturbanceStart + recoveryTime);
else
    fprintf('Recovery Time                     : System did NOT
settle within 2%% band during simulation.\n');
end
fprintf('=====
=====');

% Phase 6B – Yaw Disturbance Zoomed Plot
figure('Name','Phase 6B: Yaw Disturbance Zoomed
(Degrees)','Position',[150 150 950 520]);

% Convert radians to degrees
y_deg      = rad2deg(y);
refValue_deg = rad2deg(refValue);
band_deg    = rad2deg(band);
maxDev_deg  = rad2deg(maxDev);
peakAngle_deg = rad2deg(peakAngle);

% Zoom tightly around the pulse (0.090s - 0.098s)
t_zoom_start = 0.075;
t_zoom_end   = 0.135;
idx_zoom = (t >= t_zoom_start) & (t <= t_zoom_end);

% 1. Disturbance pulse shaded box (8ms duration: 0.090s to
0.098s)
patch([disturbanceStart disturbanceEnd disturbanceEnd
disturbanceStart], ...
      [4.95 4.95 5.05 5.05], [1 0.85 0.95], 'EdgeColor', 'none',
...

```

```

        'FaceAlpha', 0.6, 'DisplayName', 'Disturbance Pulse
(8ms)');
hold on;

% 2. +/- 2% Recovery Band (4.9 deg to 5.1 deg)
yline(refValue_deg + band_deg, ':r', 'LineWidth', 1.1,
'DisplayName', '\pm2% Recovery Band (\pm0.1^\circ)');
yline(refValue_deg - band_deg, ':r', 'LineWidth', 1.1,
'HandleVisibility', 'off');

% 3. Reference setpoint
yline(refValue_deg, 'k--', 'LineWidth', 1.2, 'DisplayName',
'Reference (5^\circ)');

% 4. True Yaw Angle
plot(t(idx_zoom), y_deg(idx_zoom), 'b', 'LineWidth', 2.0,
'DisplayName', 'True Yaw Angle \psi');

% 5. Peak deviation marker
plot(peakTime, peakAngle_deg, 'ro', 'MarkerFaceColor', 'r',
'MarkerSize', 6, ...
'DisplayName', sprintf('Peak Bump (+%.4f^\circ)',
maxDev_deg));

grid on;
xlim([t_zoom_start, t_zoom_end]);

% Millidegree scale centered around 5 deg
ylim([4.970, 5.030]);

xlabel('Time (s)', 'FontSize', 11);
ylabel('Yaw Angle \psi (^\circ)', 'FontSize', 11);
title(sprintf('Yaw Disturbance Rejection (Zoomed): Peak Bump =
+%.4f^\circ (Never leaves \pm2% band)', ...
maxDev_deg), 'FontSize', 12);

legend('Location', 'southeast', 'FontSize', 9);

%% Phase 6C – Pitch Disturbance Rejection Analysis (Kalman
Feedback)
% 1. Simulation Setup
modelName = 'Single_Axis_Pitch_Disturbance_Est_Feedback';
open_system(modelName);
sim(modelName);

```

```

% 2. Extract True Pitch Angle
data = theta;
t = data.Time;
y = data.Data(:);

% 3. Disturbance & Reference Parameters
refValue      = deg2rad(5);    % 5 deg setpoint (0.08727 rad)
disturbanceStart = 0.100;      % Aligned at 0.10s to let pitch
settle
disturbanceEnd   = 0.1035;      % 3.5ms pulse duration
band             = 0.02 * refValue; % 2% recovery band (+/-0.1
deg)

% 4. Transient Window (0.10s to 0.15s)
idx_transient = (t >= disturbanceStart) & (t <= 0.150);
t_trans      = t(idx_transient);
y_trans      = y(idx_transient);

% 5. Peak Upward Bump (Crest Detection)
dev_trans     = y_trans - refValue;
[maxDev, relIdx] = max(dev_trans);
peakTime      = t_trans(relIdx);
peakAngle     = y_trans(relIdx);

% 6. Steady-State Chatter Analysis (Post-Transient: 0.15s to
0.20s)
idx_ss        = t >= 0.150;
ss_error      = y(idx_ss) - refValue;
rms_chatter_rad = rms(ss_error);
rms_chatter_deg = rad2deg(rms_chatter_rad);
rms_chatter_pct = (rms_chatter_rad / refValue) * 100;

% 7. Degree Conversions for Plotting & Display
y_deg         = rad2deg(y);
refValue_deg  = rad2deg(refValue);
band_deg      = rad2deg(band);
maxDev_deg    = rad2deg(maxDev);
peakAngle_deg = rad2deg(peakAngle);

% 8. Display Key Metrics
fprintf('\n===== PHASE 6C – PITCH DISTURBANCE
REJECTION =====\n');
fprintf('Reference Target Angle           : %.5f rad (%.2f deg)\n',
refValue, refValue_deg);
fprintf('True Peak Bump from Target           : %.5f rad (%.3f deg)\n',
maxDev, maxDev_deg);

```

```

fprintf('Peak Bump Percentage           : %.2f %% of
reference\n', (maxDev / refValue) * 100);
fprintf('Time of Peak Bump              : %.4f s (at t = %.4f
s)\n', peakTime - disturbanceStart, peakTime);
fprintf('Steady-State RMS Chatter        : %.5f rad (%.3f deg,
%.2f%% of ref)\n', ...
        rms_chatter_rad, rms_chatter_deg, rms_chatter_pct);
fprintf('Settling Status                  : Persistent bounded
chatter (does not enter quiet steady state)\n');
fprintf('=====\n');

% 9. Full Plot with Corrected Peak Marker
figure('Name','Phase 6C: Pitch Disturbance (Full
Response)','Position',[150 150 950 520]);

% A. Disturbance pulse shaded region
patch([disturbanceStart disturbanceEnd disturbanceEnd
disturbanceStart], ...
      [0 0 7 7], [1 0.85 0.95], 'EdgeColor', 'none', ...
      'FaceAlpha', 0.6, 'DisplayName', 'Disturbance Pulse
(3.5ms)');
hold on;

% B. Reference and tolerance lines
yline(refValue_deg + band_deg, ':r', 'LineWidth', 1.0,
'DisplayName', '\pm2% Recovery Band');
yline(refValue_deg - band_deg, ':r', 'LineWidth', 1.0,
'HandleVisibility', 'off');
yline(refValue_deg, 'k--', 'LineWidth', 1.2, 'DisplayName',
'Reference (5^\circ)');

% C. True pitch trajectory
plot(t, y_deg, 'b', 'LineWidth', 1.6, 'DisplayName', 'True Pitch
Angle \theta');

% D. Corrected Peak Marker
plot(peakTime, peakAngle_deg, 'ro', 'MarkerFaceColor', 'r',
'MarkerSize', 6, ...
'DisplayName', sprintf('Peak Bump (+%.3f^\circ at %.3fs)',
maxDev_deg, peakTime));

grid on;
xlim([0, 0.2]);
ylim([0, 6.5]);

```

```

xlabel('Time (s)', 'FontSize', 11);
ylabel('Pitch Angle \theta (^{\circ})', 'FontSize', 11);
title(sprintf('Phase 6C Pitch Disturbance Rejection: Peak Bump =
+%.3f^{\circ}, RMS Chatter = %.2f%%', ...
    maxDev_deg, rms_chatter_pct), 'FontSize', 12);

legend('Location', 'southeast', 'FontSize', 9);

%% Phase 6C – Roll Disturbance Rejection Analysis (alman
Feedback)

modelName = 'Single_Axis_Roll_Disturbance_Est_Feedback'; % <--
Your model name
open_system(modelName);
sim(modelName);

data = phi;
t = data.Time;
y = data.Data(:);

% Reference and Disturbance Parameters
refValue      = deg2rad(5);    % 5 deg setpoint (0.08727 rad)
disturbanceStart = 0.100;      % 0.1s onset
disturbanceEnd   = 0.105;      % 5ms pulse duration

% Isolate post-disturbance window
idx_post = t >= disturbanceStart;
t_post   = t(idx_post);
y_post   = y(idx_post);

% Peak Deviation
deviation      = y_post - refValue;
[maxDev, maxIdx] = max(abs(deviation));
peakTime       = t_post(maxIdx);
peakAngle      = y_post(maxIdx);

% Recovery Time (2% band around 5 deg: +/-0.1 deg)
band           = 0.02 * refValue;
recovered      = false;
recoveryTime   = NaN;

for k = maxIdx:length(y_post)
    if all(abs(y_post(k:end) - refValue) <= band)
        recoveryTime = t_post(k) - disturbanceStart;
        recovered     = true;
    end
end

```

```

        break;
    end
end

% Display Metrics
fprintf('\n===== PHASE 6C – DISTURBANCE REJECTION\n\n');
fprintf('Reference Target Angle      : %.5f rad (%.2f deg)\n',
refValue, rad2deg(refValue));
fprintf('Peak Deviation from Target    : %.5f rad (%.3f deg)\n',
maxDev, rad2deg(maxDev));
fprintf('Peak Deviation Percentage      : %.2f %% of\n\n',
reference, (maxDev / refValue) * 100);
fprintf('Time to Reach Peak Deviation    : %.4f s (after pulse\n\n',
onset)\n', peakTime - disturbanceStart);

if recovered
    fprintf('Recovery Time (to 2%% band)      : %.4f s (after\n\n',
pulse onset)\n', recoveryTime);
    fprintf('Total Settling Timestamp          : %.4f s\n',
disturbanceStart + recoveryTime);
else
    fprintf('Recovery Time                        : System did NOT\n\n',
settle within 2%% band during simulation.\n');
end
fprintf('=====
=====\\n');

% Zoomed Rejection Plot (Degrees)
figure('Name','Phase 6C: Roll Disturbance (Kalman\n\n',
Feedback)', 'Position',[150 150 950 520]);

y_deg      = rad2deg(y);
refValue_deg = rad2deg(refValue);
band_deg    = rad2deg(band);
maxDev_deg  = rad2deg(maxDev);
peakAngle_deg = rad2deg(peakAngle);

t_zoom_start = 0.070;
t_zoom_end   = 0.160;
idx_zoom     = (t >= t_zoom_start) & (t <= t_zoom_end);

patch([disturbanceStart disturbanceEnd disturbanceEnd
disturbanceStart], ...
      [4.5 4.5 6.0 6.0], [1 0.85 0.95], 'EdgeColor', 'none', ...

```

```

        'FaceAlpha', 0.6, 'DisplayName', 'Disturbance Pulse (5ms,
0.015 N\cdot m)');
hold on;

yline(refValue_deg + band_deg, ':r', 'LineWidth', 1.1,
'DisplayName', '\pm 2% Recovery Band (\pm 0.1^\circ)');
yline(refValue_deg - band_deg, ':r', 'LineWidth', 1.1,
'HandleVisibility', 'off');
yline(refValue_deg, 'k--', 'LineWidth', 1.2, 'DisplayName',
'Reference (5^\circ)');

plot(t(idx_zoom), y_deg(idx_zoom), 'b', 'LineWidth', 1.8,
'DisplayName', 'True Roll Angle \phi');
plot(peakTime, peakAngle_deg, 'ro', 'MarkerFaceColor', 'r',
'MarkerSize', 6, ...
'DisplayName', sprintf('Peak Bump (+%.3f^\circ)',
maxDev_deg));

grid on;
xlim([t_zoom_start, t_zoom_end]);
ylim([4.8, max(5.4, peakAngle_deg + 0.1)]);

xlabel('Time (s)', 'FontSize', 11);
ylabel('Roll Angle \phi (^\circ)', 'FontSize', 11);
title(sprintf('Phase 6C Roll Disturbance Rejection: Peak Bump =
%.3f^\circ, Recovery Time = %.4fs', ...
maxDev_deg, recoveryTime), 'FontSize', 12);

legend('Location', 'southeast', 'FontSize', 9);

%% Phase 6C – Yaw Disturbance Rejection Analysis (alman Feedback)

modelName = 'Single_Axis_Yaw_Disturbance_Est_Feedback'; % <--
Your model name
open_system(modelName);
sim(modelName);

data = psi;
t = data.Time;
y = data.Data(:);

% Reference and Disturbance Parameters
refValue      = deg2rad(5);    % 5 deg setpoint (0.08727 rad)
disturbanceStart = 0.12;      % 0.1s onset
disturbanceEnd   = 0.1235;    % 5ms pulse duration

```

```

% Isolate post-disturbance window
idx_post = t >= disturbanceStart;
t_post    = t(idx_post);
y_post    = y(idx_post);

% Peak Deviation
deviation      = y_post - refValue;
[maxDev, maxIdx] = max(abs(deviation));
peakTime       = t_post(maxIdx);
peakAngle      = y_post(maxIdx);

% Recovery Time (2% band around 5 deg: +/-0.1 deg)
band           = 0.02 * refValue;
recovered      = false;
recoveryTime   = NaN;

for k = maxIdx:length(y_post)
    if all(abs(y_post(k:end) - refValue) <= band)
        recoveryTime = t_post(k) - disturbanceStart;
        recovered    = true;
        break;
    end
end

% Display Metrics
fprintf('\n===== PHASE 6C – DISTURBANCE REJECTION\n\n');
fprintf('Reference Target Angle      : %.5f rad (0.2f deg)\n',
refValue, rad2deg(refValue));
fprintf('Peak Deviation from Target     : %.5f rad (0.3f deg)\n',
maxDev, rad2deg(maxDev));
fprintf('Peak Deviation Percentage       : 0.2f %% of\n\n',
reference\n', (maxDev / refValue) * 100);
fprintf('Time to Reach Peak Deviation    : 0.4f s (after pulse\n\n',
onset)\n', peakTime - disturbanceStart);

if recovered
    fprintf('Recovery Time (to 2%% band)      : 0.4f s (after\n\n',
pulse onset)\n', recoveryTime);
    fprintf('Total Settling Timestamp         : 0.4f s\n',
disturbanceStart + recoveryTime);
else
    fprintf('Recovery Time                        : System did NOT\n\n',
settle within 2%% band during simulation.\n');
end

```

```

fprintf('=====
=====\\n');

% Zoomed Rejection Plot (Degrees)
figure('Name','Phase 6C: Roll Disturbance (Kalman
Feedback)','Position',[150 150 950 520]);

y_deg      = rad2deg(y);
refValue_deg = rad2deg(refValue);
band_deg    = rad2deg(band);
maxDev_deg  = rad2deg(maxDev);
peakAngle_deg = rad2deg(peakAngle);

t_zoom_start = 0.070;
t_zoom_end   = 0.160;
idx_zoom     = (t >= t_zoom_start) & (t <= t_zoom_end);

patch([disturbanceStart disturbanceEnd disturbanceEnd
disturbanceStart], ...
      [4.5 4.5 6.0 6.0], [1 0.85 0.95], 'EdgeColor', 'none', ...
      'FaceAlpha', 0.6, 'DisplayName', 'Disturbance Pulse (5ms,
0.015 N\\cdot m)');
hold on;

yline(refValue_deg + band_deg, 'r', 'LineWidth', 1.1,
'DisplayName', '\\pm2% Recovery Band (\\pm0.1^\\circ)');
yline(refValue_deg - band_deg, 'r', 'LineWidth', 1.1,
'HandleVisibility', 'off');
yline(refValue_deg, 'k--', 'LineWidth', 1.2, 'DisplayName',
'Reference (5^\\circ)');

plot(t(idx_zoom), y_deg(idx_zoom), 'b', 'LineWidth', 1.8,
'DisplayName', 'True Yaw Angle \\psi');
plot(peakTime, peakAngle_deg, 'ro', 'MarkerFaceColor', 'r',
'MarkerSize', 6, ...
'DisplayName', sprintf('Peak Bump (+%.3f^\\circ)',
maxDev_deg));

grid on;
xlim([t_zoom_start, t_zoom_end]);
ylim([4.8, max(5.4, peakAngle_deg + 0.1)]);

xlabel('Time (s)', 'FontSize', 11);
ylabel('Roll Angle \\phi (^\\circ)', 'FontSize', 11);
title(sprintf('Phase 6C Yaw Disturbance Rejection: Peak Bump =
+%.3f^\\circ, Recovery Time = %.4fs', ...

```

```

        maxDev_deg, recoveryTime), 'FontSize', 12);

legend('Location', 'southeast', 'FontSize', 9);

%% Phase 6D – Pitch Disturbance Severity Sweep (Estimate
Feedback)

modelName = 'Single_Axis_Pitch_Disturbance_Est_Feedback'; % <--
your Kalman-feedback pitch model
open_system(modelName);

% Block paths to your two disturbance Step blocks – replace with
actual paths
% (click each block, type `gcb` in Command Window to get exact
path string)
stepUpBlock = [modelName '/Pitch Axis Dynamics/Step_ON'];
stepDownBlock = [modelName '/Pitch Axis Dynamics/Step_OFF'];

baseAmplitude = 0.0003; % pitch's original 1x
disturbance (N*m)
severityMultipliers = [1, 2, 4, 8];

refValue = deg2rad(5);
disturbanceStart = 0.100; % corrected onset, past pitch's
step response
disturbanceEnd = 0.1035;
tailMargin = 0.03; % how far past disturbanceEnd
counts as "settled" window

results = table('Size', [length(severityMultipliers) 5], ...
    'VariableTypes',
    {'double','double','double','double','string'}, ...
    'VariableNames',
    {'Multiplier','Amplitude_Nm','PeakDeviation_pct', ...
    'RMS_Chatter_pct','Status'});

figure('Name','Phase 6D – Pitch Severity Sweep','Position',[100
100 1100 750]);

for i = 1:length(severityMultipliers)
    mult = severityMultipliers(i);
    amp = baseAmplitude * mult;

```

```

    % Push the scaled amplitude into both step blocks (down-step
    is negative)
    set_param(stepUpBlock, 'After', num2str(amp));
    set_param(stepDownBlock, 'After', num2str(-amp));

    simOut = sim(modelName);

    % Pull true theta – adjust variable/signal name to match your
    model
    data = theta;    % or evalin('base','theta') if using To
    Workspace
    t = data.Time;
    y = data.Data(:);

    idx_post = t >= disturbanceStart;
    t_post = t(idx_post);
    y_post = y(idx_post);

    deviation = y_post - refValue;
    [maxDev, ~] = max(abs(deviation));

    tail_window = t >= (disturbanceEnd + tailMargin);
    y_tail = y(tail_window);
    rms_chatter = sqrt(mean((y_tail - refValue).^2));

    peakPct = 100 * maxDev / refValue;
    chatterPct = 100 * rms_chatter / refValue;

    if max(abs(y)) > 3*refValue || any(isnan(y))
        status = "DIVERGED";
    elseif chatterPct > 5
        status = "UNSTABLE CHATTER";
    elseif chatterPct > 2
        status = "PERSISTENT CHATTER";
    else
        status = "SETTLES CLEANLY";
    end

    results.Multiplier(i) = mult;
    results.Amplitude_Nm(i) = amp;
    results.PeakDeviation_pct(i) = peakPct;
    results.RMS_Chatter_pct(i) = chatterPct;
    results.Status(i) = status;

    subplot(2,2,i);
    plot(t, rad2deg(y), 'b', 'LineWidth', 1.3); hold on;

```

```

        yline(rad2deg(refValue), '--k', 'Reference');
        grid on;
        xlim([0.05 0.2]);
        title(sprintf('%dx baseline (%.4g N*m): peak=%.2f%%,
chatter=%.2f%%', ...
            mult, amp, peakPct, chatterPct));
        xlabel('Time (s)'); ylabel('\theta (deg)');
    end

    fprintf('\n===== PHASE 6D – PITCH SEVERITY SWEEP
===== \n');
    disp(results)

%% Phase 6D – Roll Disturbance Severity Sweep (Estimate Feedback)

modelName = 'Single_Axis_Roll_Disturbance_Est_Feedback'; % <--
your roll estimate-feedback model
open_system(modelName);

stepUpBlock = [modelName '/Roll Axis Dynamics/Step_ON']; %
confirm via gcb on each block
stepDownBlock = [modelName '/Roll Axis Dynamics/Step_OFF'];

baseAmplitude = 0.015; % roll's original 1x
disturbance (N*m)
severityMultipliers = [1, 2, 4, 8];

refValue = deg2rad(5);
disturbanceStart = 0.100; % roll's onset
disturbanceEnd = 0.105; % roll's 5ms pulse width
tailMargin = 0.03;

results = table('Size', [length(severityMultipliers) 5], ...
    'VariableTypes',
    {'double','double','double','double','string'}, ...
    'VariableNames',
    {'Multiplier','Amplitude_Nm','PeakDeviation_pct', ...
        'RMS_Chatter_pct','Status'});

figure('Name','Phase 6D – Roll Severity Sweep','Position',[100
100 1100 750]);

for i = 1:length(severityMultipliers)
    mult = severityMultipliers(i);
    amp = baseAmplitude * mult;

```

```

set_param(stepUpBlock, 'After', num2str(amp));
set_param(stepDownBlock, 'After', num2str(-amp));

simOut = sim(modelName);

data = phi; % adjust to match your actual signal/variable
name
t = data.Time;
y = data.Data(:);

idx_post = t >= disturbanceStart;
t_post = t(idx_post);
y_post = y(idx_post);

deviation = y_post - refValue;
[maxDev, ~] = max(abs(deviation));

tail_window = t >= (disturbanceEnd + tailMargin);
y_tail = y(tail_window);
rms_chatter = sqrt(mean((y_tail - refValue).^2));

peakPct = 100 * maxDev / refValue;
chatterPct = 100 * rms_chatter / refValue;

if max(abs(y)) > 3*refValue || any(isnan(y))
    status = "DIVERGED";
elseif chatterPct > 5
    status = "UNSTABLE CHATTER";
elseif chatterPct > 2
    status = "PERSISTENT CHATTER";
else
    status = "SETTLES CLEANLY";
end

results.Multiplier(i) = mult;
results.Amplitude_Nm(i) = amp;
results.PeakDeviation_pct(i) = peakPct;
results.RMS_Chatter_pct(i) = chatterPct;
results.Status(i) = status;

subplot(2,2,i);
plot(t, rad2deg(y), 'b', 'LineWidth', 1.3); hold on;
yline(rad2deg(refValue), '--k', 'Reference');
grid on;
xlim([0.05 0.3]);

```

```

        title(sprintf('%dx baseline (%.4g N*m): peak=%.2f%%,
chatter=%.2f%%', ...
            mult, amp, peakPct, chatterPct));
        xlabel('Time (s)'); ylabel('\phi (deg)');
    end

    fprintf('\n===== PHASE 6D – ROLL SEVERITY SWEEP
===== \n');
    disp(results)

%% Phase 6D – Yaw Disturbance Severity Sweep (Estimate Feedback)

modelName = 'Single_Axis_Yaw_Disturbance_Est_Feedback'; % <--
your yaw estimate-feedback model
open_system(modelName);

stepUpBlock = [modelName '/Yaw Axis Dynamics/Step_ON']; %
confirm via gcb on each block
stepDownBlock = [modelName '/Yaw Axis Dynamics/Step_OFF'];

baseAmplitude = 0.001; % yaw's original 1x
disturbance (N*m)
severityMultipliers = [1, 2, 4, 8];

refValue = deg2rad(5);
disturbanceStart = 0.120; % yaw's revised, safer onset
disturbanceEnd = 0.1235; % yaw's pulse width
tailMargin = 0.03;

results = table('Size', [length(severityMultipliers) 5], ...
    'VariableTypes',
    {'double','double','double','double','string'}, ...
    'VariableNames',
    {'Multiplier','Amplitude_Nm','PeakDeviation_pct', ...
    'RMS_Chatter_pct','Status'});

figure('Name','Phase 6D – Yaw Severity Sweep','Position',[100 100
1100 750]);

for i = 1:length(severityMultipliers)
    mult = severityMultipliers(i);
    amp = baseAmplitude * mult;

    set_param(stepUpBlock, 'After', num2str(amp));
    set_param(stepDownBlock, 'After', num2str(-amp));

```

```

simOut = sim(modelName);

data = psi;    % adjust to match your actual signal/variable
name
t = data.Time;
y = data.Data(:);

idx_post = t >= disturbanceStart;
t_post = t(idx_post);
y_post = y(idx_post);

deviation = y_post - refValue;
[maxDev, ~] = max(abs(deviation));

tail_window = t >= (disturbanceEnd + tailMargin);
y_tail = y(tail_window);
rms_chatter = sqrt(mean((y_tail - refValue).^2));

peakPct = 100 * maxDev / refValue;
chatterPct = 100 * rms_chatter / refValue;

if max(abs(y)) > 3*refValue || any(isnan(y))
    status = "DIVERGED";
elseif chatterPct > 5
    status = "UNSTABLE CHATTER";
elseif chatterPct > 2
    status = "PERSISTENT CHATTER";
else
    status = "SETTLES CLEANLY";
end

results.Multiplier(i) = mult;
results.Amplitude_Nm(i) = amp;
results.PeakDeviation_pct(i) = peakPct;
results.RMS_Chatter_pct(i) = chatterPct;
results.Status(i) = status;

subplot(2,2,i);
plot(t, rad2deg(y), 'b', 'LineWidth', 1.3); hold on;
yline(rad2deg(refValue), '--k', 'Reference');
grid on;
xlim([0.08 0.3]);
title(sprintf('%dx baseline (%.4g N*m): peak=%.2f%%,
chatter=%.2f%%', ...
    mult, amp, peakPct, chatterPct));

```

```

        xlabel('Time (s)'); ylabel('\psi (deg)');
end

fprintf('\n===== PHASE 6D – YAW SEVERITY SWEEP\n');
disp(results)

%% Phase 6E – Simultaneous Multi-Axis Disturbance (Coupled Model,
Estimate Feedback)

modelName = 'Simultaneous_Multi_Axis_Disturbance_model'; % <--
your Phase 4/5D coupled model filename
open_system(modelName);

%% Reference amplitude for this run – change to deg2rad(57.3) for
the aggressive test
refAmplitude = deg2rad(57.3);

set_param([modelName '/r_phi'], 'After',
num2str(refAmplitude));
set_param([modelName '/r_theta'], 'After',
num2str(refAmplitude));
set_param([modelName '/r_psi'], 'After',
num2str(refAmplitude));

%% Disturbance parameters – shared onset, per-axis proportional
duration
distStart = 0.150;

rollAmp = 0.015;    rollDistEnd = 0.155;    % 5ms pulse
pitchAmp = 0.0003; pitchDistEnd = 0.1535;    % 3.5ms pulse
yawAmp = 0.001;    yawDistEnd = 0.1577;    % 7.7ms pulse

% Confirm each block path via gcb before running – these are
placeholders
set_param([modelName '/Roll Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', num2str(rollAmp));
set_param([modelName '/Roll Axis Dynamics/Step_OFF'], 'Time',
num2str(rollDistEnd), 'After', num2str(-rollAmp));

set_param([modelName '/Pitch Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', num2str(pitchAmp));
set_param([modelName '/Pitch Axis Dynamics/Step_OFF'], 'Time',
num2str(pitchDistEnd), 'After', num2str(-pitchAmp));

```

```

set_param([modelName '/Yaw Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', num2str(yawAmp));
set_param([modelName '/Yaw Axis Dynamics/Step_OFF'], 'Time',
num2str(yawDistEnd), 'After', num2str(-yawAmp));

%% Run simulation
simOut = sim(modelName);

%% Analyze all three axes
trueVarNames = {'phi', 'theta', 'psi'};
labels = {'phi', 'theta', 'psi'};

lastPulseEnd = max([rollDistEnd, pitchDistEnd, yawDistEnd]); %
latest pulse end, for tail window

results = table('Size', [3 4], ...
    'VariableTypes', {'string','double','double','string'}, ...
    'VariableNames',
    {'Axis','PeakDeviation_pct','RMS_Chatter_pct','Status'});

figure('Name','Phase 6E – Simultaneous Multi-Axis
Disturbance','Position',[100 100 1300 420]);

for i = 1:3
    data = evalin('base', trueVarNames{i}); % adjust if using
To Workspace struct instead
    t = data.Time;
    y = data.Data(:);

    idx_post = t >= distStart;
    deviation = y(idx_post) - refAmplitude;
    [maxDev, ~] = max(abs(deviation));

    tail_window = t >= (lastPulseEnd + 0.03);
    y_tail = y(tail_window);
    rms_chatter = sqrt(mean((y_tail - refAmplitude).^2));

    peakPct = 100 * maxDev / refAmplitude;
    chatterPct = 100 * rms_chatter / refAmplitude;

    if max(abs(y)) > 3*refAmplitude || any(isnan(y))
        status = "DIVERGED";
    elseif chatterPct > 5
        status = "UNSTABLE CHATTER";
    elseif chatterPct > 2
        status = "PERSISTENT CHATTER";

```

```

else
    status = "SETTLES CLEANLY";
end

results.Axis(i) = labels{i};
results.PeakDeviation_pct(i) = peakPct;
results.RMS_Chatter_pct(i) = chatterPct;
results.Status(i) = status;

subplot(1,3,i);
plot(t, rad2deg(y), 'b', 'LineWidth', 1.3); hold on;
yline(rad2deg(refAmplitude), '--k', 'Reference');
xline(distStart, ':r', 'Disturbance');
grid on;
title(sprintf('%s: peak=%.2f%%, chatter=%.2f%%', labels{i},
peakPct, chatterPct));
xlabel('Time (s)'); ylabel('deg');
end

fprintf('\n===== PHASE 6E – SIMULTANEOUS DISTURBANCE
(ref=%.1f deg) =====\n', rad2deg(refAmplitude));
disp(results)

%% Phase 7 – Attitude Magnitude Sweep, Coupled + Disturbance
(Estimate Feedback)

modelName = 'Simultaneous_Multi_Axis_Disturbance_model'; % <--
your coupled estimate-feedback model
open_system(modelName);

angleSweep_deg = [5, 10, 20, 30, 40, 50, 57.3];

labels = {'phi', 'theta', 'psi'};
trueVarNames = {'phi', 'theta', 'psi'}; % adjust to match your
To Workspace names

%% Disturbance parameters – identical to Phase 6E
distStart = 0.150;

rollAmp = 0.015;    rollDistEnd = 0.155;    % 5ms pulse
pitchAmp = 0.0003;  pitchDistEnd = 0.1535;   % 3.5ms pulse
yawAmp = 0.001;     yawDistEnd = 0.1577;     % 7.7ms pulse

lastPulseEnd = max([rollDistEnd, pitchDistEnd, yawDistEnd]);

```

```

% Set disturbance blocks once – same for every angle in the sweep
set_param([modelName '/Roll Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', num2str(rollAmp));
set_param([modelName '/Roll Axis Dynamics/Step_OFF'], 'Time',
num2str(rollDistEnd), 'After', num2str(-rollAmp));

set_param([modelName '/Pitch Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', num2str(pitchAmp));
set_param([modelName '/Pitch Axis Dynamics/Step_OFF'], 'Time',
num2str(pitchDistEnd), 'After', num2str(-pitchAmp));

set_param([modelName '/Yaw Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', num2str(yawAmp));
set_param([modelName '/Yaw Axis Dynamics/Step_OFF'], 'Time',
num2str(yawDistEnd), 'After', num2str(-yawAmp));

%% Results table
results = table('Size', [length(angleSweep_deg)*3 6], ...
'VariableTypes',
{'double','string','double','double','double','string'}, ...
'VariableNames',
{'Angle_deg','Axis','Overshoot_pct','SettlingTime_s','RMS_Chatter
_pct','Status'});

rowIdx = 1;

for a = 1:length(angleSweep_deg)
    refAmplitude = deg2rad(angleSweep_deg(a));

    set_param([modelName '/r_phi'], 'After',
num2str(refAmplitude));
    set_param([modelName '/r_theta'], 'After',
num2str(refAmplitude));
    set_param([modelName '/r_psi'], 'After',
num2str(refAmplitude));

    simOut = sim(modelName);

    for i = 1:3
        data = evalin('base', trueVarNames{i});
        t = data.Time;
        y = data.Data(:);

        try
            info = stepinfo(y, t, refAmplitude);
            overshoot = info.Overshoot;

```

```

        settlingTime = info.SettlingTime;
    catch
        overshoot = NaN;
        settlingTime = NaN;
    end

    % Post-disturbance chatter, same method as 6D/6E
    tail_window = t >= (lastPulseEnd + 0.03);
    y_tail = y(tail_window);
    rms_chatter = sqrt(mean((y_tail - refAmplitude).^2));
    chatterPct = 100 * rms_chatter / refAmplitude;

    if isnan(settlingTime) || max(abs(y)) > 3*refAmplitude
        status = "UNACCEPTABLE (unbounded/never settles)";
    elseif overshoot > 20 || chatterPct > 5
        status = "UNACCEPTABLE (overshoot/chatter)";
    elseif overshoot > 5 || chatterPct > 2
        status = "DEGRADING";
    else
        status = "ACCEPTABLE";
    end

    results.Angle_deg(rowIdx) = angleSweep_deg(a);
    results.Axis(rowIdx) = labels{i};
    results.Overshoot_pct(rowIdx) = overshoot;
    results.SettlingTime_s(rowIdx) = settlingTime;
    results.RMS_Chatter_pct(rowIdx) = chatterPct;
    results.Status(rowIdx) = status;
    rowIdx = rowIdx + 1;
end
end

fprintf('\n===== PHASE 7 – ANGLE SWEEP + DISTURBANCE\n');
disp(results)

%% Phase 7 (isolation test) – Coupled, NO Disturbance, Attitude Sweep

modelName = 'Simultaneous_Multi_Axis_Disturbance_model'; % <--
your coupled estimate-feedback model
open_system(modelName);

angleSweep_deg = [5, 10, 57.3];

labels = {'phi', 'theta', 'psi'};

```

```

trueVarNames = {'phi', 'theta', 'psi'}; % adjust to match your
To Workspace names

%% Zero out the disturbance blocks – same blocks as 6E, amplitude
set to 0
distStart = 0.150;
rollDistEnd = 0.155;
pitchDistEnd = 0.1535;
yawDistEnd = 0.1577;
lastPulseEnd = max([rollDistEnd, pitchDistEnd, yawDistEnd]);

set_param([modelName '/Roll Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', '0');
set_param([modelName '/Roll Axis Dynamics/Step_OFF'], 'Time',
num2str(rollDistEnd), 'After', '0');

set_param([modelName '/Pitch Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', '0');
set_param([modelName '/Pitch Axis Dynamics/Step_OFF'], 'Time',
num2str(pitchDistEnd), 'After', '0');

set_param([modelName '/Yaw Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', '0');
set_param([modelName '/Yaw Axis Dynamics/Step_OFF'], 'Time',
num2str(yawDistEnd), 'After', '0');

%% Results table
results = table('Size', [length(angleSweep_deg)*3 6], ...
'VariableTypes',
{'double','string','double','double','double','string'}, ...
'VariableNames',
{'Angle_deg','Axis','Overshoot_pct','SettlingTime_s','RMS_Chatter
_pct','Status'});

rowIdx = 1;

for a = 1:length(angleSweep_deg)
    refAmplitude = deg2rad(angleSweep_deg(a));

    set_param([modelName '/r_phi'], 'After',
num2str(refAmplitude));
    set_param([modelName '/r_theta'], 'After',
num2str(refAmplitude));
    set_param([modelName '/r_psi'], 'After',
num2str(refAmplitude));

```

```

simOut = sim(modelName);

for i = 1:3
    data = evalin('base', trueVarNames{i});
    t = data.Time;
    y = data.Data(:);

    try
        info = stepinfo(y, t, refAmplitude);
        overshoot = info.Overshoot;
        settlingTime = info.SettlingTime;
    catch
        overshoot = NaN;
        settlingTime = NaN;
    end

    tail_window = t >= (lastPulseEnd + 0.03);
    y_tail = y(tail_window);
    rms_chatter = sqrt(mean((y_tail - refAmplitude).^2));
    chatterPct = 100 * rms_chatter / refAmplitude;

    if isnan(settlingTime) || max(abs(y)) > 3*refAmplitude
        status = "UNACCEPTABLE (unbounded/never settles)";
    elseif overshoot > 20 || chatterPct > 5
        status = "UNACCEPTABLE (overshoot/chatter)";
    elseif overshoot > 5 || chatterPct > 2
        status = "DEGRADING";
    else
        status = "ACCEPTABLE";
    end

    results.Angle_deg(rowIdx) = angleSweep_deg(a);
    results.Axis(rowIdx) = labels{i};
    results.Overshoot_pct(rowIdx) = overshoot;
    results.SettlingTime_s(rowIdx) = settlingTime;
    results.RMS_Chatter_pct(rowIdx) = chatterPct;
    results.Status(rowIdx) = status;
    rowIdx = rowIdx + 1;
end
end

fprintf('\n===== PHASE 7 (ISOLATION) – COUPLED, NO
DISTURBANCE =====\n');
disp(results)

```

```

%% Phase 8 – Combined Stress Test: Angle x Disturbance Severity
(Coupled, Estimate Feedback)

modelName = 'Simultaneous_Multi_Axis_Disturbance_model'; % <--
your coupled estimate-feedback model
open_system(modelName);

angleSweep_deg = [5, 10, 20, 30, 57.3, 90, 120, 180];
severityMultipliers = [1, 2, 4, 8, 16, 32, 64];

labels = {'phi', 'theta', 'psi'};
trueVarNames = {'phi', 'theta', 'psi'};

% Base disturbance parameters (1x, from Phase 6A)
rollAmpBase = 0.015;
pitchAmpBase = 0.0003;
yawAmpBase = 0.001;

distStart = 0.150;
rollDistEnd = 0.155;
pitchDistEnd = 0.1535;
yawDistEnd = 0.1577;
lastPulseEnd = max([rollDistEnd, pitchDistEnd, yawDistEnd]);

nAngles = length(angleSweep_deg);
nSev = length(severityMultipliers);
nRows = nAngles * nSev * 3;

results = table('Size', [nRows 7], ...
    'VariableTypes',
    {'double','double','string','double','double','double','string'},
    ...
    'VariableNames',
    {'Angle_deg','Severity_x','Axis','Overshoot_pct','SettlingTime_s',
    , ...
    'RMS_Chatter_pct','Status'});

rowIdx = 1;

for s = 1:nSev
    mult = severityMultipliers(s);

    % Scale disturbance amplitude for this severity level
    set_param([modelName '/Roll Axis Dynamics/Step_ON'], 'Time',
    num2str(distStart), 'After', num2str(rollAmpBase*mult));

```

```

    set_param([modelName '/Roll Axis Dynamics/Step_OFF'], 'Time',
num2str(rollDistEnd), 'After', num2str(-rollAmpBase*mult));

    set_param([modelName '/Pitch Axis Dynamics/Step_ON'],
'Time', num2str(distStart), 'After',
num2str(pitchAmpBase*mult));
    set_param([modelName '/Pitch Axis Dynamics/Step_OFF'],
'Time', num2str(pitchDistEnd), 'After', num2str(-
pitchAmpBase*mult));

    set_param([modelName '/Yaw Axis Dynamics/Step_ON'], 'Time',
num2str(distStart), 'After', num2str(yawAmpBase*mult));
    set_param([modelName '/Yaw Axis Dynamics/Step_OFF'], 'Time',
num2str(yawDistEnd), 'After', num2str(-yawAmpBase*mult));

    for a = 1:nAngles
        refAmplitude = deg2rad(angleSweep_deg(a));

        set_param([modelName '/r_phi'], 'After',
num2str(refAmplitude));
        set_param([modelName '/r_theta'], 'After',
num2str(refAmplitude));
        set_param([modelName '/r_psi'], 'After',
num2str(refAmplitude));

        try
            simOut = sim(modelName);
        catch
            % Simulation itself failed (e.g. numerical blow-up) –
record as diverged
            for i = 1:3
                results.Angle_deg(rowIdx) = angleSweep_deg(a);
                results.Severity_x(rowIdx) = mult;
                results.Axis(rowIdx) = labels{i};
                results.Overshoot_pct(rowIdx) = NaN;
                results.SettlingTime_s(rowIdx) = NaN;
                results.RMS_Chatter_pct(rowIdx) = NaN;
                results.Status(rowIdx) = "DIVERGED (sim failed)";
                rowIdx = rowIdx + 1;
            end
            continue
        end

        for i = 1:3
            data = evalin('base', trueVarNames{i});
            t = data.Time;

```

```

y = data.Data(:);

try
    info = stepinfo(y, t, refAmplitude);
    overshoot = info.Overshoot;
    settlingTime = info.SettlingTime;
catch
    overshoot = NaN;
    settlingTime = NaN;
end

tail_window = t >= (lastPulseEnd + 0.03);
y_tail = y(tail_window);
rms_chatter = sqrt(mean((y_tail - refAmplitude).^2));
chatterPct = 100 * rms_chatter / refAmplitude;

if any(isnan(y)) || max(abs(y)) > 3*refAmplitude
    status = "DIVERGED";
elseif isnan(settlingTime)
    if chatterPct > 5
        status = "UNACCEPTABLE (unstable chatter)";
    else
        status = "DEGRADING (never settles,
bounded)";
    end
elseif overshoot > 20 || chatterPct > 5
    status = "UNACCEPTABLE";
elseif overshoot > 5 || chatterPct > 2
    status = "DEGRADING";
else
    status = "ACCEPTABLE";
end

results.Angle_deg(rowIdx) = angleSweep_deg(a);
results.Severity_x(rowIdx) = mult;
results.Axis(rowIdx) = labels{i};
results.Overshoot_pct(rowIdx) = overshoot;
results.SettlingTime_s(rowIdx) = settlingTime;
results.RMS_Chatter_pct(rowIdx) = chatterPct;
results.Status(rowIdx) = status;
rowIdx = rowIdx + 1;
end
end
end

```

```

fprintf('\n===== PHASE 8 – COMBINED ANGLE x SEVERITY
STRESS TEST =====\n');
disp(results)

% Quick filter: show only rows where anything actually diverged
divergedRows = results(contains(results.Status, "DIVERGED"), :);
fprintf('\n--- Rows where DIVERGENCE occurred ---\n');
disp(divergedRows)

% Quick filter: pitch-only view for easy reading
pitchRows = results(results.Axis == "theta", :);
fprintf('\n--- Pitch (theta) results only ---\n');
disp(pitchRows)

```
